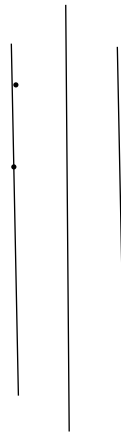




Tribhuvan University
Faculty of Humanities and Social Sciences
Saraswati Multiple Campus
Lekhnath Marg, Thamel
Kathmandu



Lab Report
Computer Graphics and Animation (CACS305)

Submitted by:

Name: Diwas Lamichhane

Grade: BCA 5th

TU Reg No: 6-2-32-12-2022

Submitted to:

Er. Manoj Giri

Table of Contents

1.	2D Transformation.....	1
1.1	Program to Implement 2D Translation	1
1.2	Program to Implement 2D Scaling Through the Origin	3
1.3	Program to Implement 2D Scaling Through an Arbitrary (Fixed) Point	5
1.4	Program to Implement 2D Rotation in Clockwise Direction.....	8
1.5	Program to Implement 2D Rotation Anti-clockwise Direction	10
1.6	Program to Implement 2D Rotation Through an Arbitrary Point in Clockwise Direction	12
1.7	Program to Implement 2D Rotation Through an Arbitrary Point in Anti-clockwise direction	14
1.8	Program to Implement 2D Reflection Over X-axis.....	17
1.9	Program to Implement 2D Reflection Over Y-axis.....	20
1.10	Program to Implement 2D Reflection Over the Origin.....	23
1.11	Program to Implement 2D Reflection Through Line Parallel to X-axis	25
1.12	Program to Implement 2D Reflection Through Line Parallel to Y-axis	27
1.13	Program to Implement 2D Reflection Through an Arbitrary Line ($y=mx+c$)	29
1.14	Program to Implement 2D Shearing in the X-direction	32
1.15	Program to Implement 2D Shearing in the Y-direction	34
1.16	Program to Implement 2D Shearing in the XY-direction	37
2.	Circle Drawing Algorithm	39
2.1	Program to Implement General Circle Drawing Algorithm	39
2.2	Program to Implement Midpoint Circle Drawing Algorithm	40
2.3	Program to Implement Midpoint Ellipse Drawing Algorithm	42
3.	Line Drawing Algorithm	46
3.1	Program to Implement DDA Line Drawing Algorithm.....	46
3.2	Program to Implement Bresenham's Line Drawing Algorithm (BLA)	48
4.	Area Filling Algorithm	50
4.1	Program to Implement Boundary Fill Algorithm	50
4.2	Program to Implement Flood Fill Algorithm.....	52
5.	Clipping.....	54
5.1	Program to Implement Line Clipping	54
6.	3D Transformation.....	58
6.1	Program to Implement 3D Translation	58
6.2	Program to Implement 3D Scaling Through Origin	61

6.3	Program to Implement 3D Rotation	64
6.4	Program to Implement 3D Reflection Through X-axis	67
6.5	Program to Implement 3D Reflection Through Y-axis.....	70
6.6	Program to Implement 3D Shearing in XY Direction	73

1. 2D Transformation

1.1 Program to Implement 2D Translation

Source Code:

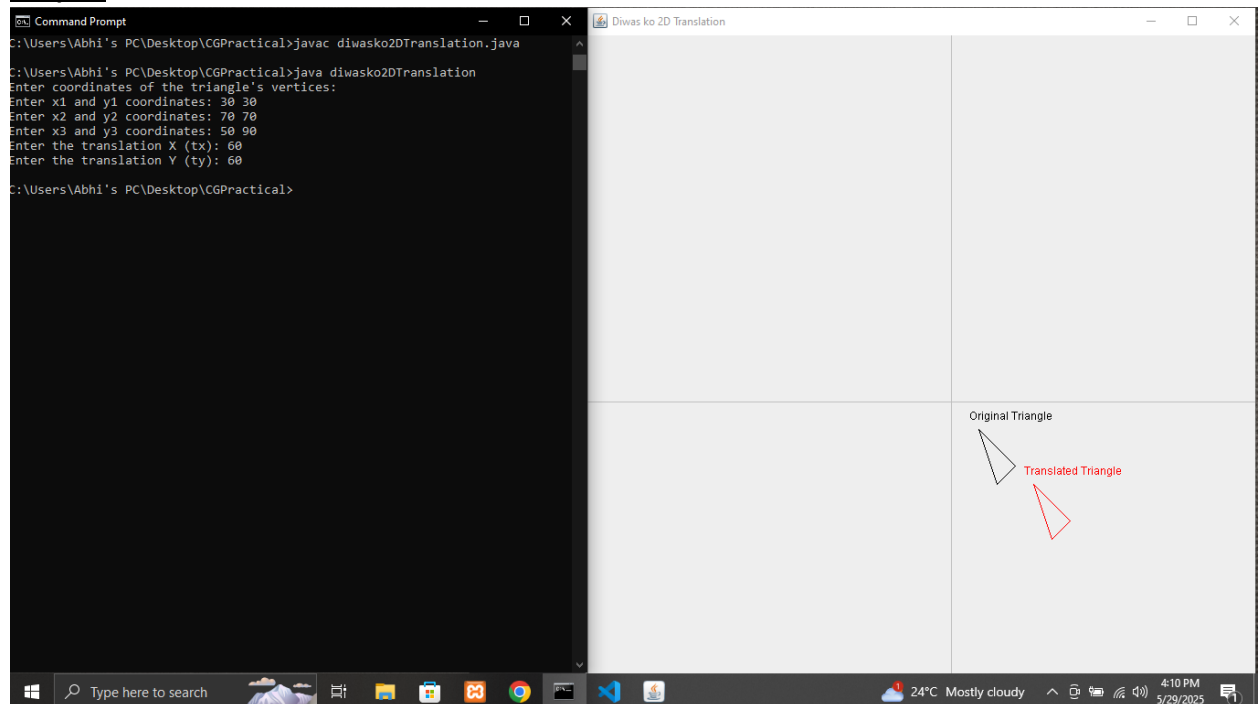
```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;
public class diwasko2DTranslation extends JPanel {
    private int[] x, y;
    private int diwas_tx, diwas_ty;
    public diwasko2DTranslation(int[] x, int[] y, int diwas_tx, int diwas_ty) {
        this.x = x;
        this.y = y;
        this.diwas_tx = diwas_tx;
        this.diwas_ty = diwas_ty;
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.LIGHT_GRAY);
        g.drawLine(0, 400, 800, 400); // X-axis
        g.drawLine(400, 0, 400, 800); // Y-axis
        int[] adjustedX = new int[3];
        int[] adjustedY = new int[3];
        for (int i = 0; i < 3; i++) {
            adjustedX[i] = x[i] + 400;
            adjustedY[i] = y[i] + 400;
        }
        g.setColor(Color.BLACK);
        g.drawPolygon(adjustedX, adjustedY, 3);
        g.drawString("Original Triangle", adjustedX[0] - 10, adjustedY[0] - 10);
        int[] translatedX = new int[3];
        int[] translatedY = new int[3];
        for (int i = 0; i < 3; i++) {
            translatedX[i] = adjustedX[i] + diwas_tx;
            translatedY[i] = adjustedY[i] + diwas_ty;
        }
        g.setColor(Color.RED);
        g.drawPolygon(translatedX, translatedY, 3);
        g.drawString("Translated Triangle", translatedX[0] - 10, translatedY[0] - 10);
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter coordinates of the triangle's vertices:");
        int[] x = new int[3];
```

```

int[] y = new int[3];
for (int i = 0; i < 3; i++) {
    System.out.print("Enter x" + (i + 1) + " and y" + (i + 1) + " coordinates: ");
    x[i] = scanner.nextInt();
    y[i] = scanner.nextInt();
}
System.out.print("Enter the translation X (tx): ");
int diwas_tx = scanner.nextInt();
System.out.print("Enter the translation Y (ty): ");
int diwas_ty = scanner.nextInt();
JFrame frame = new JFrame("Diwas ko 2D Translation");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(800, 800);
diwasko2DTranslation plot = new diwasko2DTranslation(x, y, diwas_tx, diwas_ty);
frame.add(plot);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
}
}

```

Output:



1.2 Program to Implement 2D Scaling Through the Origin

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class scalingOrigin_2D extends JPanel {
    private int[] x, y;
    private double sx, sy;
    public scalingOrigin_2D(int[] x, int[] y, double sx, double sy) {
        this.x = x;
        this.y = y;
        this.sx = sx;
        this.sy = sy;
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.LIGHT_GRAY);
        g.drawLine(0, height / 2, width, height / 2); // X-axis
        g.drawLine(width / 2, 0, width / 2, height); // Y-axis

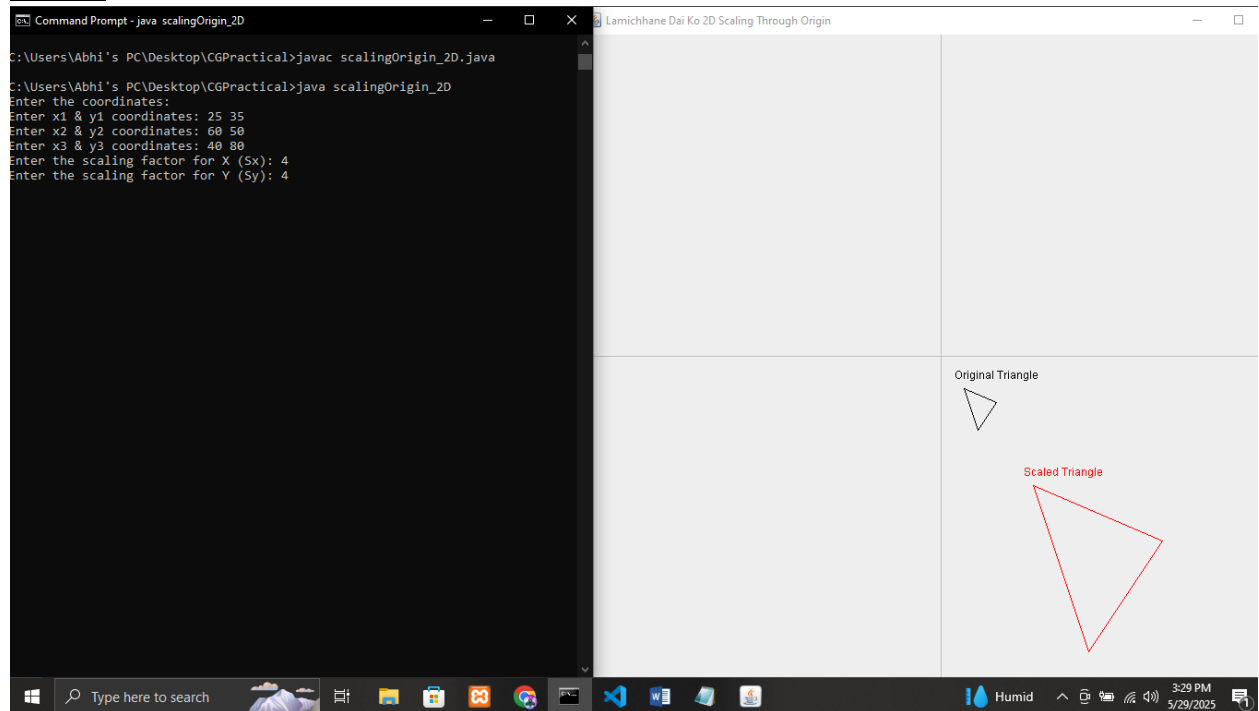
        int[] adjustedX = new int[3];
        int[] adjustedY = new int[3];
        for (int i = 0; i < 3; i++) {
            adjustedX[i] = x[i] + width / 2;
            adjustedY[i] = height / 2 + y[i];
        }
        g.setColor(Color.BLACK);
        g.drawPolygon(adjustedX, adjustedY, 3);
        g.drawString("Original Triangle", adjustedX[0] - 10, adjustedY[0] - 10);
        int[] scaledX = new int[3];
        int[] scaledY = new int[3];
        for (int i = 0; i < 3; i++) {
            scaledX[i] = (int) (x[i] * sx) + width / 2;
            scaledY[i] = height / 2 + (int) (y[i] * sy);
        }
        g.setColor(Color.RED);
        g.drawPolygon(scaledX, scaledY, 3);
        g.drawString("Scaled Triangle", scaledX[0] - 10, scaledY[0] - 10);
    }
}
```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int[] x = new int[3];
    int[] y = new int[3];
    System.out.println("Enter the coordinates:");
    for (int i = 0; i < 3; i++) {
        System.out.print("Enter x" + (i + 1) + " & y" + (i + 1) + " coordinates: ");
        x[i] = scanner.nextInt();
        y[i] = scanner.nextInt();
    }
    System.out.print("Enter the scaling factor for X (Sx): ");
    double sx = scanner.nextDouble();
    System.out.print("Enter the scaling factor for Y (Sy): ");
    double sy = scanner.nextDouble();
    JFrame frame = new JFrame("Lamichhane Dai Ko 2D Scaling Through Origin");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(800, 600);
    scalingOrigin_2D plot = new scalingOrigin_2D(x, y, sx, sy);
    frame.add(plot);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
}
}

```

Output:



1.3 Program to Implement 2D Scaling Through an Arbitrary (Fixed) Point

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class arbitraryPointWalaScaling extends Frame {
    int diwasx1, diwasy1, diwasx2, diwasy2, diwasx3, diwasy3;
    float diwassx, diwassy;
    int diwastx, diwasty;
    int diwaspx1, diwaspx2, diwaspy1, diwaspy2, diwaspx3, diwaspy3;

    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;

    public arbitraryPointWalaScaling() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter x1: ");
        diwasx1 = sc.nextInt();
        System.out.println("Enter y1: ");
        diwasy1 = sc.nextInt();
        System.out.println("Enter x2: ");
        diwasx2 = sc.nextInt();
        System.out.println("Enter y2: ");
        diwasy2 = sc.nextInt();
        System.out.println("Enter x3: ");
        diwasx3 = sc.nextInt();
        System.out.println("Enter y3: ");
        diwasy3 = sc.nextInt();
        System.out.println("Enter scaling factor (Sx): ");
        diwassx = sc.nextFloat();
        System.out.println("Enter scaling factor (Sy): ");
        diwassy = sc.nextFloat();
        System.out.println("Enter arbitrary point tx: ");
        diwastx = sc.nextInt();
        System.out.println("Enter arbitrary point ty: ");
        diwasty = sc.nextInt();
        sc.close();
        diwaspx1 = (int)(diwasx1 * diwassx + diwastx * (1 - diwassx));
        diwaspy1 = (int)(diwasy1 * diwassy + diwasty * (1 - diwassy));
        diwaspx2 = (int)(diwasx2 * diwassx + diwastx * (1 - diwassx));
        diwaspy2 = (int)(diwasy2 * diwassy + diwasty * (1 - diwassy));
        diwaspx3 = (int)(diwasx3 * diwassx + diwastx * (1 - diwassx));
        diwaspy3 = (int)(diwasy3 * diwassy + diwasty * (1 - diwassy));
    }
}
```



```

this.setTitle("Arbitrary Point ko through wala scaling 2D ma");
this.setLayout(null);
this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
this.setVisible(true);

this.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent we) {
        System.exit(0);
    }
});
}

private Point toAWTCoords(int x, int y) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    return new Point(centerX + x, centerY - y);
}

public void paint(Graphics g) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;

    g.setColor(Color.BLACK);
    g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);

    g.drawLine(0, centerY, getWidth(), centerY);
    g.drawLine(centerX, 0, centerX, getHeight());

    g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
    g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
    g.drawLine(centerX - 3, 10, centerX, 0);
    g.drawLine(centerX + 3, 10, centerX, 0);

    g.setColor(Color.BLUE);
    Point p1_orig_awt = toAWTCoords(diwasx1, diwasy1);
    Point p2_orig_awt = toAWTCoords(diwasx2, diwasy2);
    Point p3_orig_awt = toAWTCoords(diwasx3, diwasy3);
    g.drawLine(p1_orig_awt.x, p1_orig_awt.y, p2_orig_awt.x, p2_orig_awt.y);
    g.drawLine(p2_orig_awt.x, p2_orig_awt.y, p3_orig_awt.x, p3_orig_awt.y);
    g.drawLine(p3_orig_awt.x, p3_orig_awt.y, p1_orig_awt.x, p1_orig_awt.y);
    g.drawString("before scaling", p1_orig_awt.x, p1_orig_awt.y - 5);

    g.setColor(Color.RED);
    Point p1_scaled_awt = toAWTCoords(diwaspx1, diwaspy1);

```

```

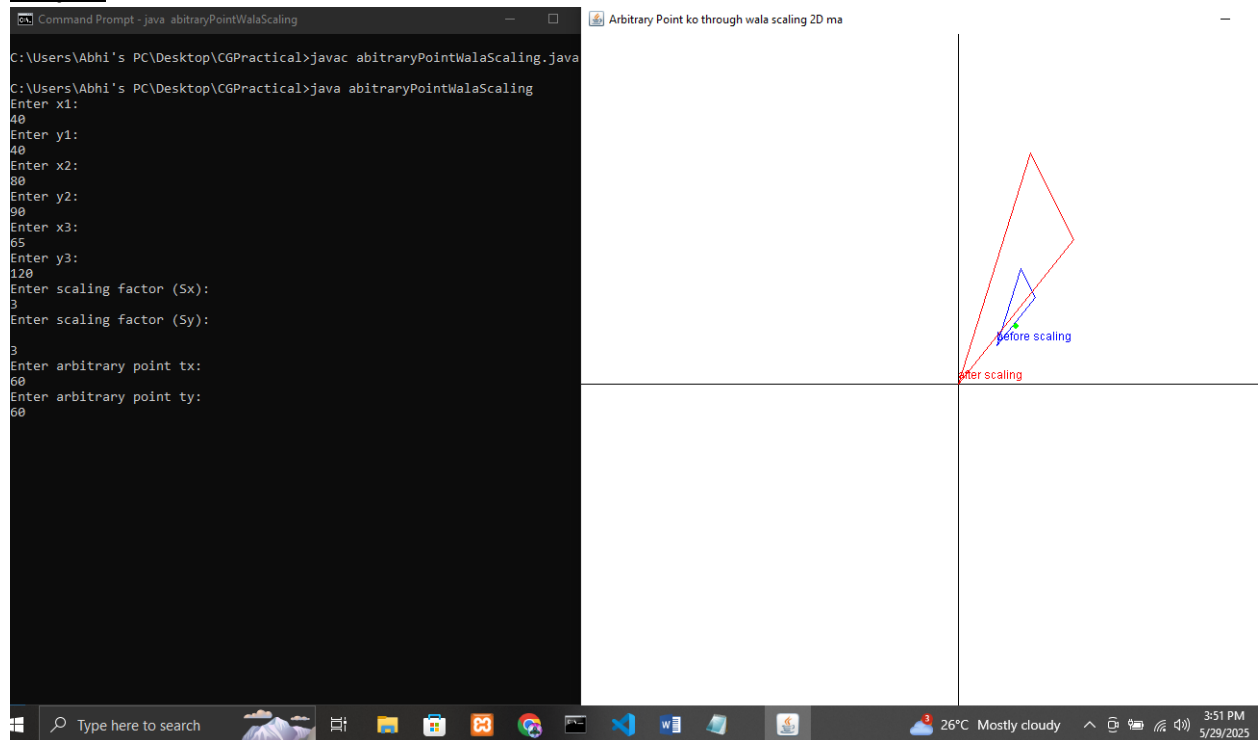
    Point p2_scaled_awt = toAWTCords(diwaspx2, diwaspy2);
    Point p3_scaled_awt = toAWTCords(diwaspx3, diwaspy3);
    g.drawLine(p1_scaled_awt.x, p1_scaled_awt.y, p2_scaled_awt.x, p2_scaled_awt.y);
    g.drawLine(p2_scaled_awt.x, p2_scaled_awt.y, p3_scaled_awt.x, p3_scaled_awt.y);
    g.drawLine(p3_scaled_awt.x, p3_scaled_awt.y, p1_scaled_awt.x, p1_scaled_awt.y);
    g.drawString("after scaling", p1_scaled_awt.x, p1_scaled_awt.y - 5);

    g.setColor(Color.GREEN);
    Point arbitrary_point_awt = toAWTCords(diwastx, diwasty);
    g.fillOval(arbitrary_point_awt.x - 3, arbitrary_point_awt.y - 3, 6, 6);
}

public static void main(String[] args) {
    new abitraryPointWalaScaling();
}
}

```

Output:



1.4 Program to Implement 2D Rotation in Clockwise Direction

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

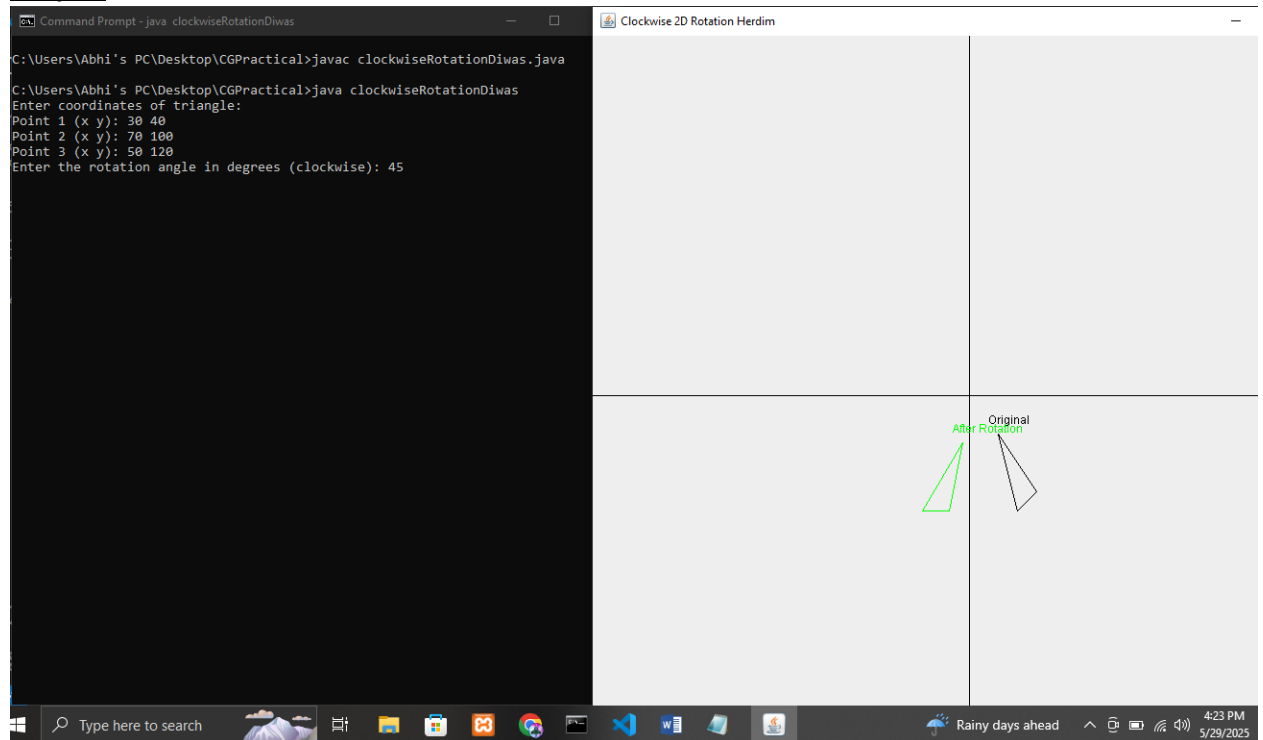
public class clockwiseRotationDiwas extends JPanel {
    private int[] diwasx, diwasy;
    private double theta;
    public clockwiseRotationDiwas(int[] diwasx, int[] diwasy, double theta) {
        this.diwasx = diwasx;
        this.diwasy = diwasy;
        this.theta = Math.toRadians(theta);
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int w = getWidth();
        int h = getHeight();
        g.setColor(Color.BLACK);
        g.drawLine(0, h / 2, w, h / 2); // X-axis
        g.drawLine(w / 2, 0, w / 2, h); // Y-axis
        int[] adjustedX = new int[3];
        int[] adjustedY = new int[3];
        for (int i = 0; i < 3; i++) {
            adjustedX[i] = diwasx[i] + w / 2;
            adjustedY[i] = h / 2 + diwasy[i]; // Adjusted by adding
        }
        g.setColor(Color.BLACK);
        g.drawPolygon(adjustedX, adjustedY, 3);
        g.drawString("Original", adjustedX[0] - 10, adjustedY[0] - 10);
        int[] rotatedX = new int[3];
        int[] rotatedY = new int[3];
        for (int i = 0; i < 3; i++) {
            rotatedX[i] = (int) ((diwasx[i] * Math.cos(theta)) - (diwasy[i] * Math.sin(theta))) + w / 2;
            rotatedY[i] = h / 2 + (int) ((diwasx[i] * Math.sin(theta)) + (diwasy[i] * Math.cos(theta)));
        }
        g.setColor(Color.GREEN);
        g.drawPolygon(rotatedX, rotatedY, 3);
        g.drawString("After Rotation", rotatedX[0] - 10, rotatedY[0] - 10);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] diwas_x = new int[3];
```

```

int[] diwas_y = new int[3];
System.out.println("Enter coordinates of triangle:");
for (int i = 0; i < 3; i++) {
    System.out.print("Point " + (i + 1) + " (x y): ");
    diwas_x[i] = sc.nextInt();
    diwas_y[i] = sc.nextInt();
}
System.out.print("Enter the rotation angle in degrees (clockwise): ");
double theta = sc.nextDouble();
JFrame frame = new JFrame("Clockwise 2D Rotation Herdim");
clockwiseRotationDiwas panel = new clockwiseRotationDiwas(diwas_x, diwas_y, theta);
frame.add(panel);
frame.setSize(800, 800);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Output:



1.5 Program to Implement 2D Rotation Anti-clockwise Direction

Source Code:

```
import javax.swing.*.*;
import java.awt.*.*;
import java.util.Scanner;

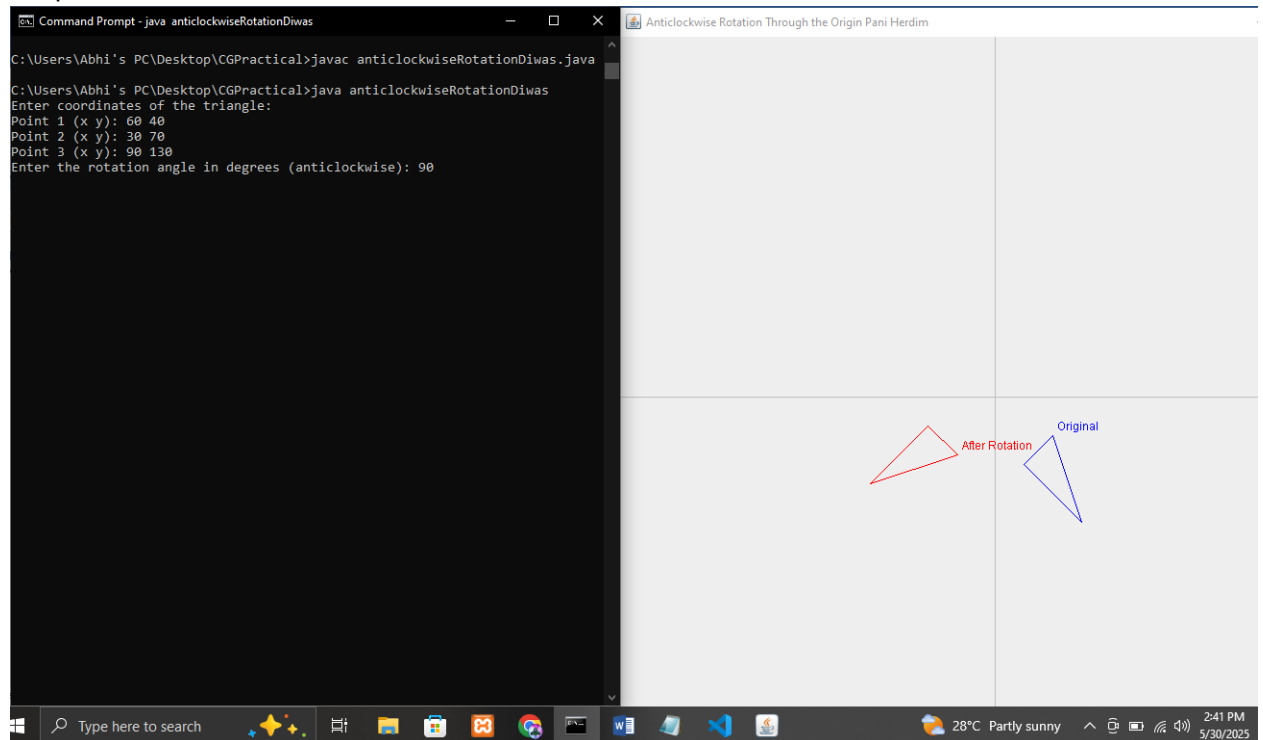
public class anticlockwiseRotationDiwas extends JPanel {
    private int[] x;
    private int[] y;
    private double angle;
    public anticlockwiseRotationDiwas(int[] x, int[] y, double angle) {
        this.x = x;
        this.y = y;
        this.angle = Math.toRadians(angle);
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.LIGHT_GRAY);
        g.drawLine(0, height / 2, width, height / 2); // X-axis
        g.drawLine(width / 2, 0, width / 2, height); // Y-axis
        int[] adjustedX = new int[3];
        int[] adjustedY = new int[3];
        for (int i = 0; i < 3; i++) {
            adjustedX[i] = x[i] + width / 2;
            adjustedY[i] = y[i] + height / 2;
        }
        g.setColor(Color.BLUE);
        g.drawPolygon(adjustedX, adjustedY, 3);
        g.drawString("Original", adjustedX[0] + 5, adjustedY[0] - 5);
        int[] rotatedX = new int[3];
        int[] rotatedY = new int[3];
        for (int i = 0; i < 3; i++) {
            rotatedX[i] = (int) (x[i] * Math.cos(angle) - y[i] * Math.sin(angle)) + width / 2;
            rotatedY[i] = (int) (x[i] * Math.sin(angle) + y[i] * Math.cos(angle)) + height / 2;
        }
        g.setColor(Color.RED);
        g.drawPolygon(rotatedX, rotatedY, 3);
        g.drawString("After Rotation", rotatedX[0] + 5, rotatedY[0] - 5);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] x = new int[3];
```

```

int[] y = new int[3];
System.out.println("Enter coordinates of the triangle: ");
for (int i = 0; i < 3; i++) {
    System.out.print("Point " + (i + 1) + " (x y): ");
    x[i] = sc.nextInt();
    y[i] = sc.nextInt();
}
System.out.print("Enter the rotation angle in degrees (anticlockwise): ");
double angle = sc.nextDouble();
JFrame frame = new JFrame("Anticlockwise Rotation Through the Origin Pani Herdim");
anticlockwiseRotationDiwas panel = new anticlockwiseRotationDiwas(x, y, angle);
frame.add(panel);
frame.setSize(800, 800);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Output:



1.6 Program to Implement 2D Rotation Through an Arbitrary Point in Clockwise Direction

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class clkwiseFixedPoint_diwas extends Frame {
    int[] diwasey_x = {100, 150, 125};
    int[] diwasey_y = {100, 100, 50};
    double theta;
    int xr, yr;
    public clkwiseFixedPoint_diwas(double angle, int xr, int yr) {
        this.theta = Math.toRadians(angle);
        this.xr = xr;
        this.yr = yr;
        setTitle("Rotation about Arbitrary(Fixed) Point - Clockwise @Diwas Lamichhane");
        setSize(600, 600);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.BLUE);
        g.drawPolygon(diwasey_x, diwasey_y, 3);
        int originalCentroidX = 0;
        int originalCentroidY = 0;
        for (int i = 0; i < 3; i++) {
            originalCentroidX += diwasey_x[i];
            originalCentroidY += diwasey_y[i];
        }
        originalCentroidX /= 3;
        originalCentroidY /= 3;
        g.setColor(Color.BLUE);
        g.drawString("Original", originalCentroidX + 10, originalCentroidY);
        int[] rx = new int[3];
        int[] ry = new int[3];
        for (int i = 0; i < 3; i++) {
            int tx = diwasey_x[i] - xr;
            int ty = diwasey_y[i] - yr;
            rx[i] = (int)(tx * Math.cos(theta) + ty * Math.sin(theta)) + xr;
            ry[i] = (int)(-tx * Math.sin(theta) + ty * Math.cos(theta)) + yr;
        }
        g.setColor(Color.RED);
        g.drawPolygon(rx, ry, 3);
        int rotatedCentroidX = 0;
        int rotatedCentroidY = 0;
        for (int i = 0; i < 3; i++) {
            rotatedCentroidX += rx[i];
```

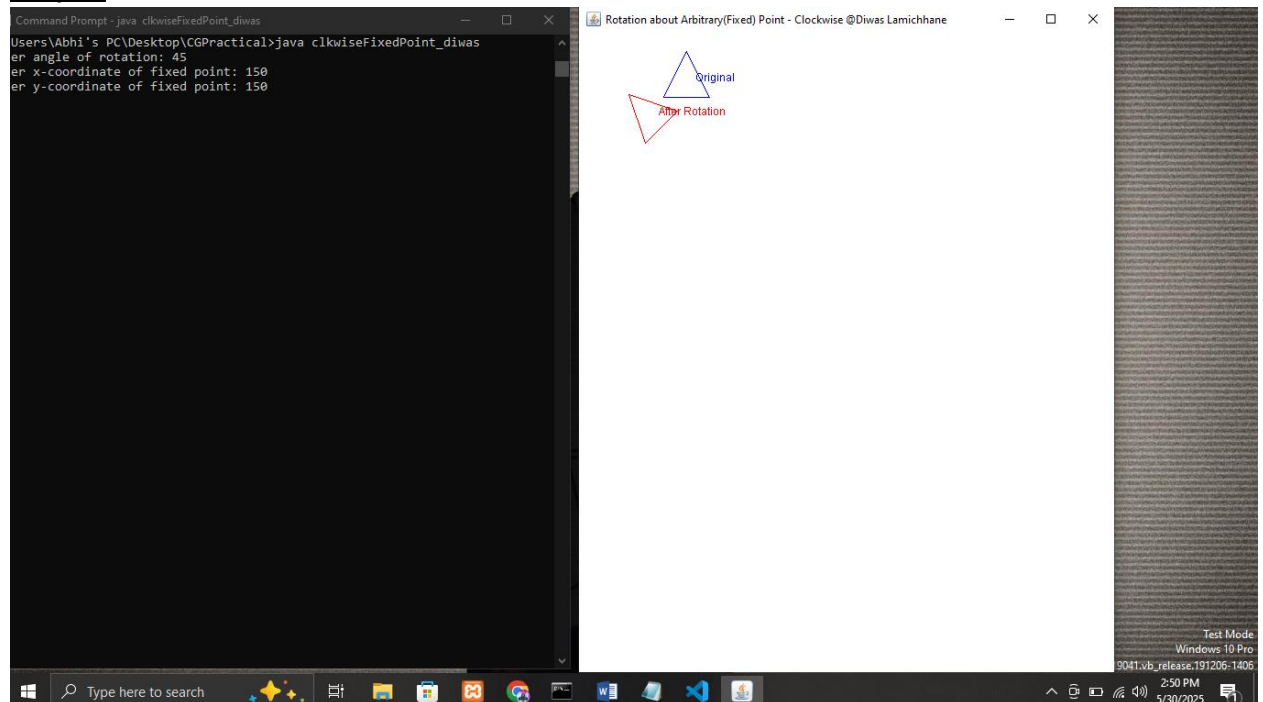
```

        rotatedCentroidY += ry[i];
    }
    rotatedCentroidX /= 3;
    rotatedCentroidY /= 3;
    g.setColor(Color.RED);
    g.drawString("After Rotation", rotatedCentroidX + 10, rotatedCentroidY);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter angle of rotation: ");
    double angle = sc.nextDouble();
    System.out.print("Enter x-coordinate of fixed point: ");
    int xr = sc.nextInt();
    System.out.print("Enter y-coordinate of fixed point: ");
    int yr = sc.nextInt();
    sc.close();
    new clkwiseFixedPoint_diwas(angle, xr, yr);
}
}

```

Output:



1.7 Program to Implement 2D Rotation Through an Arbitrary Point in Anti-clockwise direction

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class fixedPointRotationAnticlkDiwas extends Frame {
    int diwasx1, diwasy1, diwasx2, diwasy2, diwasx3, diwasy3;
    int diwasxp, diwasyp;
    int diwaspx1, diwaspx2, diwaspy1, diwaspy2, diwaspx3, diwaspy3;
    double angle;
    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;
    public fixedPointRotationAnticlkDiwas() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter x1 ");
        diwasx1 = sc.nextInt();
        System.out.println("Enter y1 ");
        diwasy1 = sc.nextInt();
        System.out.println("Enter x2 ");
        diwasx2 = sc.nextInt();
        System.out.println("Enter y2 ");
        diwasy2 = sc.nextInt();
        System.out.println("Enter x3 ");
        diwasx3 = sc.nextInt();
        System.out.println("Enter y3 ");
        diwasy3 = sc.nextInt();
        System.out.println("Enter pivot point xp ");
        diwasxp = sc.nextInt();
        System.out.println("Enter pivot point yp ");
        diwasyp = sc.nextInt();
        System.out.println("Enter rotation angle ");
        angle = Math.toRadians(sc.nextDouble());
        sc.close();
        diwasey_RotateTriangle();
        this.setTitle("Triangle Rotation about Arbitrary Point in Anti-Clockwise Direction @Diwas");
        this.setLayout(null);
        this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
        this.setVisible(true);
        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }
}
```

```

}
private void diwasey_RotateTriangle() {
    int diwasx1t = diwasx1 - diwasxp;
    int diwasy1t = diwasy1 - diwasyp;
    int diwasx2t = diwasx2 - diwasxp;
    int diwasy2t = diwasy2 - diwasyp;
    int diwasx3t = diwasx3 - diwasxp;
    int diwasy3t = diwasy3 - diwasyp;

    diwaspx1 = (int)(diwasx1t * Math.cos(angle) - diwasy1t * Math.sin(angle));
    diwaspy1 = (int)(diwasx1t * Math.sin(angle) + diwasy1t * Math.cos(angle));
    diwaspx2 = (int)(diwasx2t * Math.cos(angle) - diwasy2t * Math.sin(angle));
    diwaspy2 = (int)(diwasx2t * Math.sin(angle) + diwasy2t * Math.cos(angle));
    diwaspx3 = (int)(diwasx3t * Math.cos(angle) - diwasy3t * Math.sin(angle));
    diwaspy3 = (int)(diwasx3t * Math.sin(angle) + diwasy3t * Math.cos(angle));

    diwaspx1 += diwasxp;
    diwaspy1 += diwasyp;
    diwaspx2 += diwasxp;
    diwaspy2 += diwasyp;
    diwaspx3 += diwasxp;
    diwaspy3 += diwasyp;
}
private Point toAWTCoords(int x, int y) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    return new Point(centerX + x, centerY - y);
}
public void paint(Graphics g) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;

    g.setColor(Color.BLACK);
    g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);
    g.drawLine(0, centerY, getWidth(), centerY);
    g.drawLine(centerX, 0, centerX, getHeight());
    g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
    g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
    g.drawLine(centerX - 3, 10, centerX, 0);
    g.drawLine(centerX + 3, 10, centerX, 0);
    g.setColor(Color.BLUE);

    Point p1_orig_awt = toAWTCoords(diwasx1, diwasy1);
    Point p2_orig_awt = toAWTCoords(diwasx2, diwasy2);

```

```

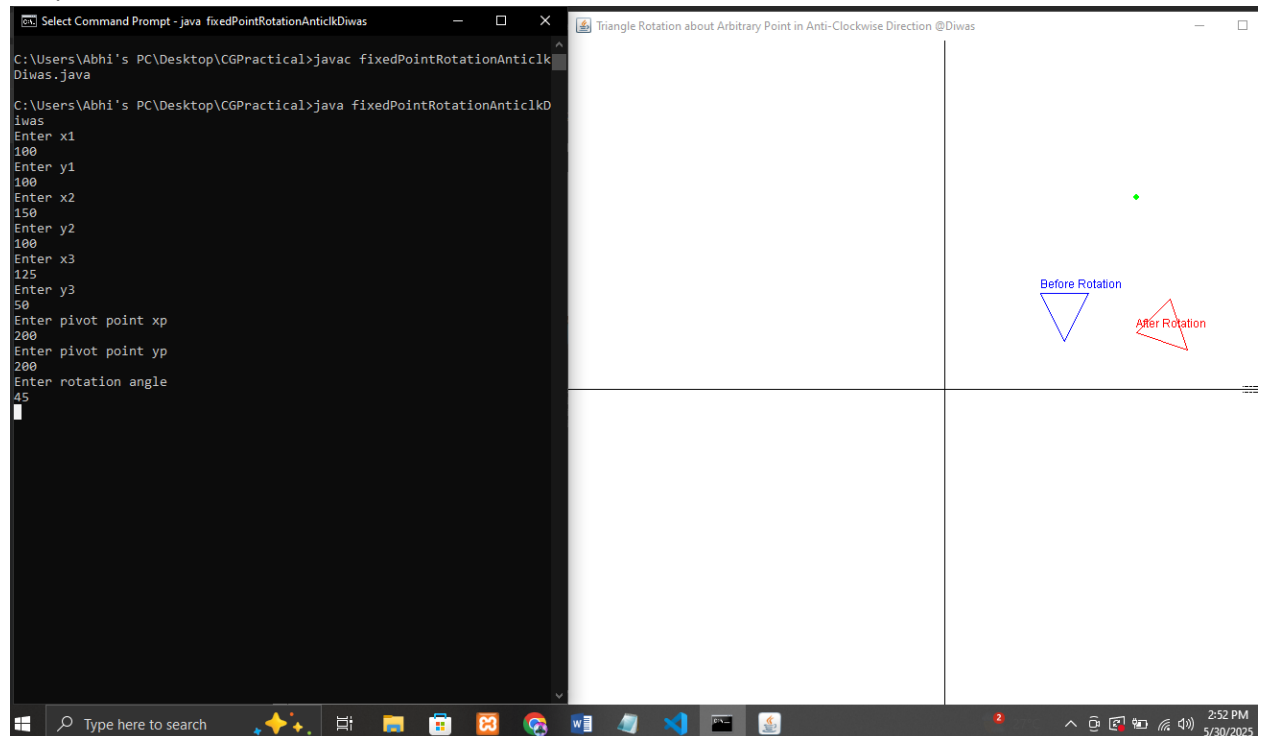
Point p3_orig_awt = toAWTCords(diwasx3, diwasy3);
g.drawLine(p1_orig_awt.x, p1_orig_awt.y, p2_orig_awt.x, p2_orig_awt.y);
g.drawLine(p2_orig_awt.x, p2_orig_awt.y, p3_orig_awt.x, p3_orig_awt.y);
g.drawLine(p3_orig_awt.x, p3_orig_awt.y, p1_orig_awt.x, p1_orig_awt.y);
g.drawString("Before Rotation", p1_orig_awt.x, p1_orig_awt.y - 5);

g.setColor(Color.RED);
Point p1_rotated_awt = toAWTCords(diwaspx1, diwaspy1);
Point p2_rotated_awt = toAWTCords(diwaspx2, diwaspy2);
Point p3_rotated_awt = toAWTCords(diwaspx3, diwaspy3);
g.drawLine(p1_rotated_awt.x, p1_rotated_awt.y, p2_rotated_awt.x, p2_rotated_awt.y);
g.drawLine(p2_rotated_awt.x, p2_rotated_awt.y, p3_rotated_awt.x, p3_rotated_awt.y);
g.drawLine(p3_rotated_awt.x, p3_rotated_awt.y, p1_rotated_awt.x, p1_rotated_awt.y);
g.drawString("After Rotation", p1_rotated_awt.x, p1_rotated_awt.y - 5);

g.setColor(Color.GREEN);
Point pivot_awt = toAWTCords(diwasxp, diwasyp);
g.fillOval(pivot_awt.x - 3, pivot_awt.y - 3, 6, 6);
}
public static void main(String[] args) {
    new fixedPointRotationAntickDiwas();
}
}

```

Output:



1.8 Program to Implement 2D Reflection Over X-axis

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class reflectionOverX_diwasdada extends Frame {
    int dbossx1, dbossy1, dbossx2, dbossy2, dbossx3, dbossy3;
    int dbossrx1, dbossry1, dbossrx2, dbossry2, dbossrx3, dbossry3;

    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;

    public reflectionOverX_diwasdada() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter x1 ");
        dbossx1 = sc.nextInt();
        System.out.println("Enter y1 ");
        dbossy1 = sc.nextInt();
        System.out.println("Enter x2 ");
        dbossx2 = sc.nextInt();
        System.out.println("Enter y2 ");
        dbossy2 = sc.nextInt();
        System.out.println("Enter x3 ");
        dbossx3 = sc.nextInt();
        System.out.println("Enter y3 ");
        dbossy3 = sc.nextInt();
        sc.close();

        dbossrx1 = dbossx1;
        dbossry1 = -dbossy1;
        dbossrx2 = dbossx2;
        dbossry2 = -dbossy2;
        dbossrx3 = dbossx3;
        dbossry3 = -dbossy3;

        this.setTitle("DBOSS Reflection Through X-axis");
        this.setLayout(null);
        this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
        this.setVisible(true);

        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }
}
```

```

    }
    });
}

private Point toAWTCoords(int x, int y) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    return new Point(centerX + x, centerY - y);
}

public void paint(Graphics g) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;

    g.setColor(Color.BLACK);
    g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);

    g.drawLine(0, centerY, getWidth(), centerY);
    g.drawLine(centerX, 0, centerX, getHeight());

    g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
    g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
    g.drawLine(centerX - 3, 10, centerX, 0);
    g.drawLine(centerX + 3, 10, centerX, 0);

    g.setColor(Color.BLUE);
    Point p1_orig_awt = toAWTCoords(dbossx1, dbossy1);
    Point p2_orig_awt = toAWTCoords(dbossx2, dbossy2);
    Point p3_orig_awt = toAWTCoords(dbossx3, dbossy3);
    g.drawLine(p1_orig_awt.x, p1_orig_awt.y, p2_orig_awt.x, p2_orig_awt.y);
    g.drawLine(p2_orig_awt.x, p2_orig_awt.y, p3_orig_awt.x, p3_orig_awt.y);
    g.drawLine(p3_orig_awt.x, p3_orig_awt.y, p1_orig_awt.x, p1_orig_awt.y);
    g.drawString("Before Reflection", p1_orig_awt.x, p1_orig_awt.y - 5);

    g.setColor(Color.RED);
    Point p1_reflected_awt = toAWTCoords(dbossrx1, dbossry1);
    Point p2_reflected_awt = toAWTCoords(dbossrx2, dbossry2);
    Point p3_reflected_awt = toAWTCoords(dbossrx3, dbossry3);
    g.drawLine(p1_reflected_awt.x, p1_reflected_awt.y, p2_reflected_awt.x,
p2_reflected_awt.y);
    g.drawLine(p2_reflected_awt.x, p2_reflected_awt.y, p3_reflected_awt.x,
p3_reflected_awt.y);
    g.drawLine(p3_reflected_awt.x, p3_reflected_awt.y, p1_reflected_awt.x,
p1_reflected_awt.y);

```

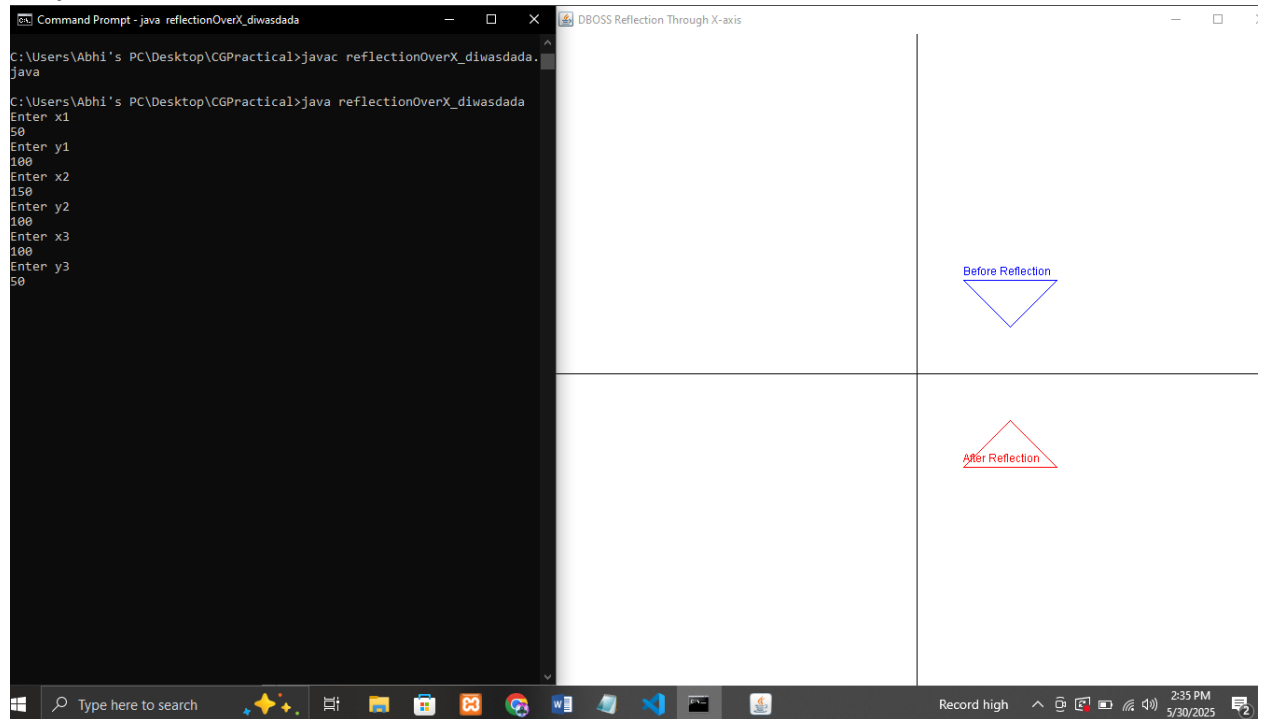
```

        g.drawString("After Reflection", p1_reflected_awt.x, p1_reflected_awt.y - 5);
    }

    public static void main(String[] args) {
        new reflectionOverX_diwasdada();
    }
}

```

Output:



1.9 Program to Implement 2D Reflection Over Y-axis

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class reflectionOverY_diwasdada extends Frame {
    int kanxi_x1, kanxi_y1, kanxi_x2, kanxi_y2, kanxi_x3, kanxi_y3;
    int kanxi_rx1, kanxi_ry1, kanxi_rx2, kanxi_ry2, kanxi_rx3, kanxi_ry3;

    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;

    public reflectionOverY_diwasdada() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter x1");
        kanxi_x1 = sc.nextInt();
        System.out.println("Enter y1");
        kanxi_y1 = sc.nextInt();
        System.out.println("Enter x2");
        kanxi_x2 = sc.nextInt();
        System.out.println("Enter y2");
        kanxi_y2 = sc.nextInt();
        System.out.println("Enter x3");
        kanxi_x3 = sc.nextInt();
        System.out.println("Enter y3");
        kanxi_y3 = sc.nextInt();
        sc.close();

        kanxi_rx1 = -kanxi_x1;
        kanxi_ry1 = kanxi_y1;
        kanxi_rx2 = -kanxi_x2;
        kanxi_ry2 = kanxi_y2;
        kanxi_rx3 = -kanxi_x3;
        kanxi_ry3 = kanxi_y3;

        this.setTitle("Diwas Lamichhane -> Reflection Through Y-axis");
        this.setLayout(null);
        this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
        this.setVisible(true);

        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }
}
```

```

    }
    });
}

private Point toAWTCoords(int x, int y) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    return new Point(centerX + x, centerY - y);
}

public void paint(Graphics g) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;

    g.setColor(Color.BLACK);
    g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);

    g.drawLine(0, centerY, getWidth(), centerY);
    g.drawLine(centerX, 0, centerX, getHeight());

    g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
    g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
    g.drawLine(centerX - 3, 10, centerX, 0);
    g.drawLine(centerX + 3, 10, centerX, 0);

    g.setColor(Color.RED);
    Point p1_orig_awt = toAWTCoords(kanxi_x1, kanxi_y1);
    Point p2_orig_awt = toAWTCoords(kanxi_x2, kanxi_y2);
    Point p3_orig_awt = toAWTCoords(kanxi_x3, kanxi_y3);
    g.drawLine(p1_orig_awt.x, p1_orig_awt.y, p2_orig_awt.x, p2_orig_awt.y);
    g.drawLine(p2_orig_awt.x, p2_orig_awt.y, p3_orig_awt.x, p3_orig_awt.y);
    g.drawLine(p3_orig_awt.x, p3_orig_awt.y, p1_orig_awt.x, p1_orig_awt.y);
    g.drawString("Before Reflection", p1_orig_awt.x, p1_orig_awt.y - 5);

    g.setColor(Color.RED);
    Point p1_reflected_awt = toAWTCoords(kanxi_rx1, kanxi_ry1);
    Point p2_reflected_awt = toAWTCoords(kanxi_rx2, kanxi_ry2);
    Point p3_reflected_awt = toAWTCoords(kanxi_rx3, kanxi_ry3);
    g.drawLine(p1_reflected_awt.x, p1_reflected_awt.y, p2_reflected_awt.x,
p2_reflected_awt.y);
    g.drawLine(p2_reflected_awt.x, p2_reflected_awt.y, p3_reflected_awt.x,
p3_reflected_awt.y);
    g.drawLine(p3_reflected_awt.x, p3_reflected_awt.y, p1_reflected_awt.x,
p1_reflected_awt.y);

```



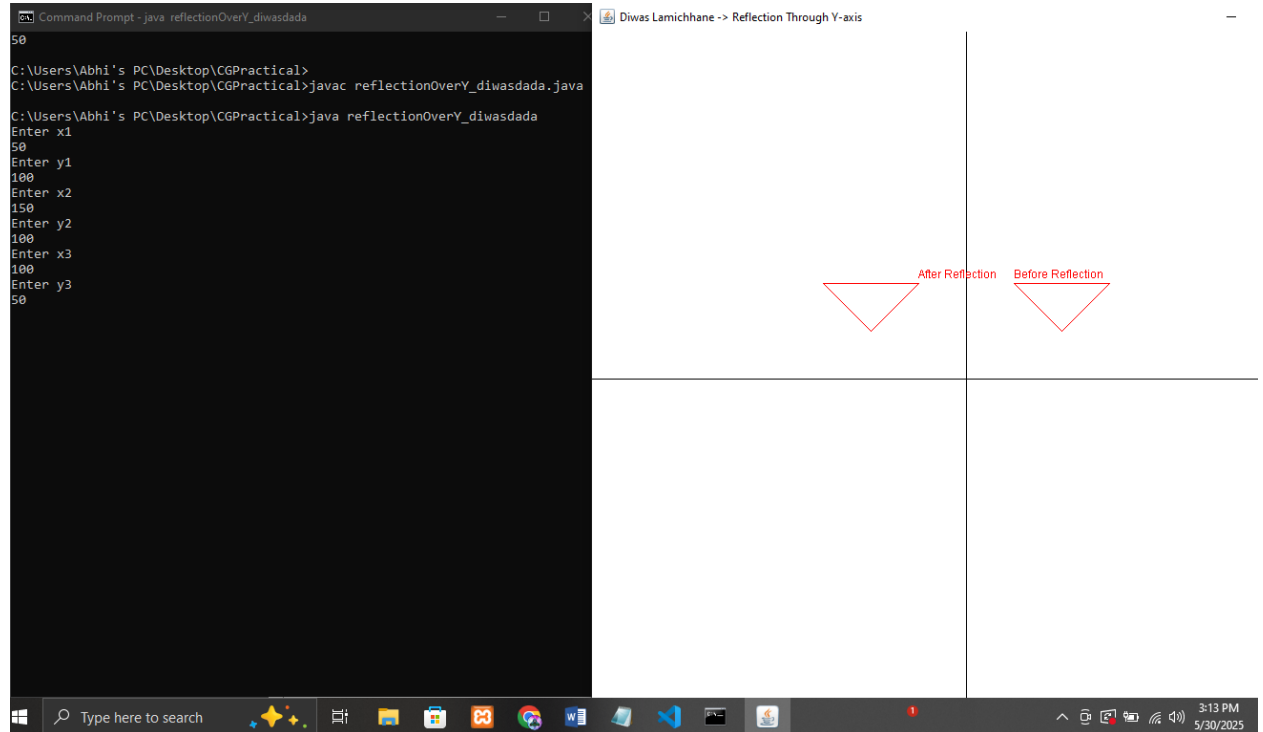
```

        g.drawString("After Reflection", p1_reflected_awt.x, p1_reflected_awt.y - 5);
    }

    public static void main(String[] args) {
        new reflectionOverY_diwasdada();
    }
}

```

Output:



1.10 Program to Implement 2D Reflection Over the Origin

Source Code:

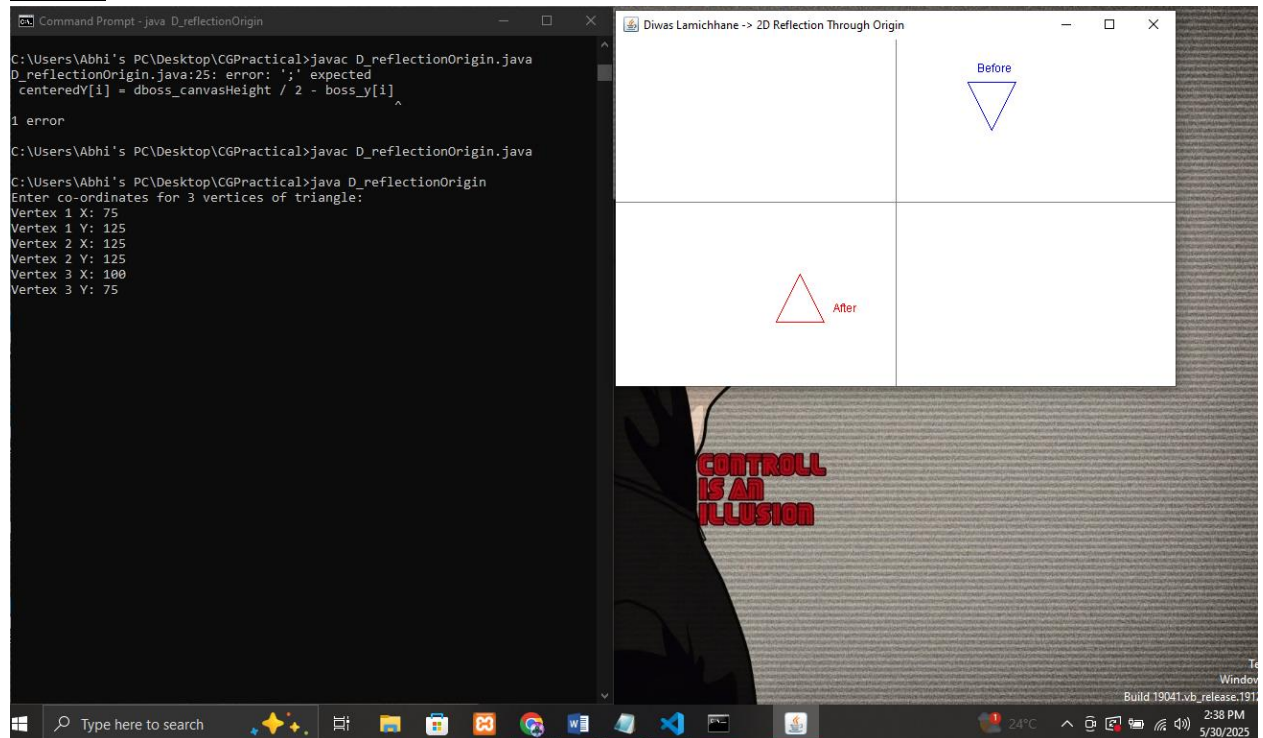
```
import java.awt.*;
import java.util.Scanner;
public class D_reflectionOrigin extends Frame {
    int[] boss_x = new int[3];
    int[] boss_y = new int[3];
    int dboss_canvasWidth = 600;
    int dboss_canvasHeight = 400;
    public D_reflectionOrigin(int[] boss_xInput, int[] boss_yInput) {
        for (int i = 0; i < 3; i++) {
            boss_x[i] = boss_xInput[i];
            boss_y[i] = boss_yInput[i];
        }
        setTitle("Diwas Lamichhane -> 2D Reflection Through Origin");
        setSize(dboss_canvasWidth, dboss_canvasHeight);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.GRAY);
        g.drawLine(0, dboss_canvasHeight / 2, dboss_canvasWidth, dboss_canvasHeight / 2);
        g.drawLine(dboss_canvasWidth / 2, 0, dboss_canvasWidth / 2, dboss_canvasHeight); // Y-axis
        int[] centeredX = new int[3];
        int[] centeredY = new int[3];
        for (int i = 0; i < 3; i++) {
            centeredX[i] = dboss_canvasWidth / 2 + boss_x[i];
            centeredY[i] = dboss_canvasHeight / 2 - boss_y[i];
        }
        g.setColor(Color.BLUE);
        g.drawPolygon(centeredX, centeredY, 3);
        g.setColor(Color.BLUE);
        g.drawString("Before", centeredX[0] + 10, centeredY[0] - 10);
        int[] reflectX = new int[3];
        int[] reflectY = new int[3];
        for (int i = 0; i < 3; i++) {
            reflectX[i] = dboss_canvasWidth / 2 - boss_x[i];
            reflectY[i] = dboss_canvasHeight / 2 + boss_y[i];
        }
        g.setColor(Color.RED);
        g.drawPolygon(reflectX, reflectY, 3);
        g.setColor(Color.RED);
        g.drawString("After", reflectX[0] + 10, reflectY[0] - 10);
    }
    public static void main(String[] args) {
```

```

Scanner diwas_input = new Scanner(System.in);
int[] userX = new int[3];
int[] userY = new int[3];
System.out.println("Enter co-ordinates for 3 vertices of triangle:");
for (int i = 0; i < 3; i++) {
    System.out.print("Vertex " + (i + 1) + " X: ");
    userX[i] = diwas_input.nextInt();
    System.out.print("Vertex " + (i + 1) + " Y: ");
    userY[i] = diwas_input.nextInt();
}
diwas_input.close();
new D_reflectionOrigin(userX, userY);
}
}

```

Output:



1.11 Program to Implement 2D Reflection Through Line Parallel to X-axis

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class DiwasrefParallel_to_X extends Frame {
    int[] deboss_x = new int[3];
    int[] deboss_y = new int[3];
    int deboss_lineY;
    public DiwasrefParallel_to_X(int[] inputX, int[] inputY, int lineY) {
        for (int i = 0; i < 3; i++) {
            deboss_x[i] = inputX[i];
            deboss_y[i] = inputY[i];
        }
        this.deboss_lineY = lineY;
        setTitle("Reflection Parallel to X-Axis (y = " + deboss_lineY + ") -> Diwas Lamichhane");
        setSize(600, 400);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.GRAY);
        g.drawLine(0, deboss_lineY, getWidth(), deboss_lineY);

        g.setColor(Color.BLUE);
        g.drawPolygon(deboss_x, deboss_y, 3);
        g.setColor(Color.BLUE);
        g.drawString("Before", deboss_x[0] + 10, deboss_y[0] - 10);

        int[] deboss_reflectY = new int[3];
        for (int i = 0; i < 3; i++) {
            deboss_reflectY[i] = 2 * deboss_lineY - deboss_y[i];
        }

        g.setColor(Color.RED);
        g.drawPolygon(deboss_x, deboss_reflectY, 3);
        g.setColor(Color.RED);
        g.drawString("After", deboss_x[0] + 10, deboss_reflectY[0] - 10);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int[] deboss_userX = new int[3];
        int[] deboss_userY = new int[3];

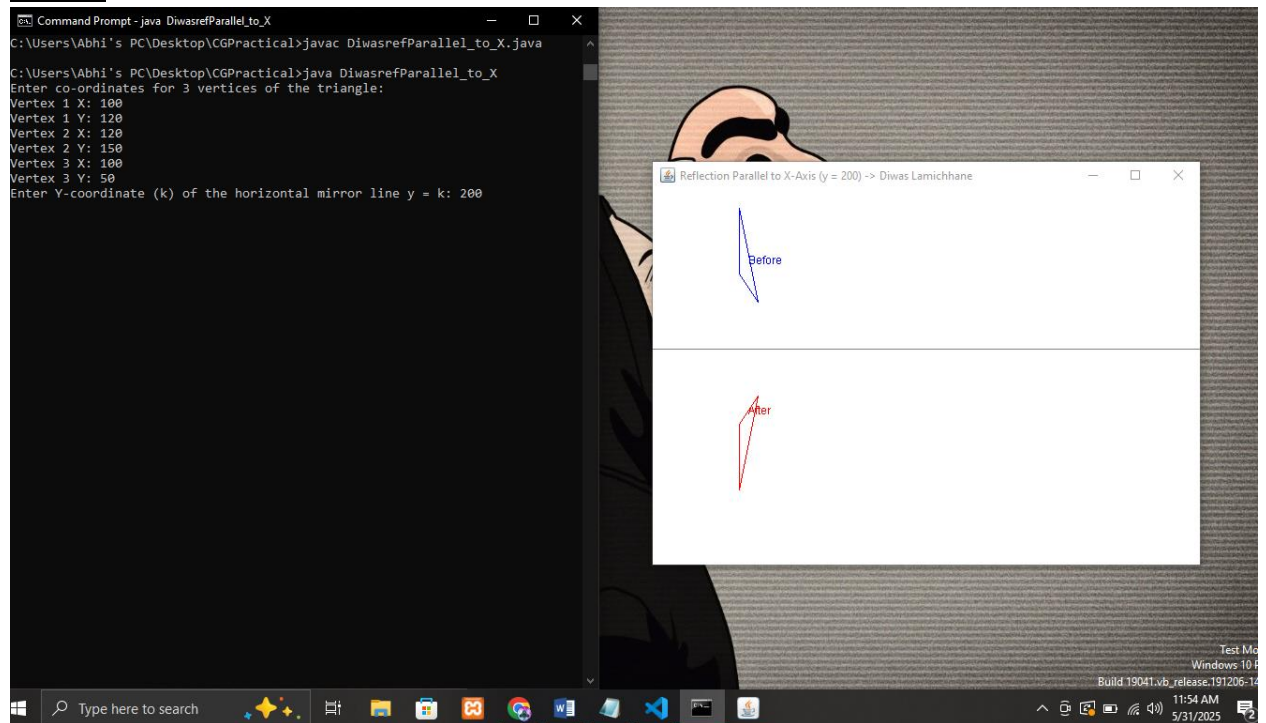
        System.out.println("Enter co-ordinates for 3 vertices of the triangle:");
```

```

for (int i = 0; i < 3; i++) {
    System.out.print("Vertex " + (i + 1) + " X: ");
    deboss_userX[i] = scan.nextInt();
    System.out.print("Vertex " + (i + 1) + " Y: ");
    deboss_userY[i] = scan.nextInt();
}
System.out.print("Enter Y-coordinate (k) of the horizontal mirror line y = k: ");
int deboss_k = scan.nextInt();
scan.close();
new DiwasrefParallel_to_X(deboss_userX, deboss_userY, deboss_k);
}
}

```

Output:



1.12 Program to Implement 2D Reflection Through Line Parallel to Y-axis

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class Diwas_refParallel_to_Y extends Frame {
    int[] diwas_x = new int[3];
    int[] diwas_y = new int[3];
    int diwas_lineX;
    int margin = 40;
    public Diwas_refParallel_to_Y(int[] inputX, int[] inputY, int lineX) {
        for (int i = 0; i < 3; i++) {
            diwas_x[i] = inputX[i];
            diwas_y[i] = inputY[i];
        }
        this.diwas_lineX = lineX;
        setTitle("Reflection Parallel to Y-Axis (x = " + diwas_lineX + ") -> Diwas Lamichhane");
        setSize(600, 500);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.GRAY);
        g.drawLine(diwas_lineX, 0, diwas_lineX, getHeight());

        g.setColor(Color.BLUE);
        g.drawPolygon(diwas_x, diwas_y, 3);
        g.setColor(Color.BLUE);
        g.drawString("Original", diwas_x[0] + 10, diwas_y[0] - 10);

        int[] diwas_reflectX = new int[3];
        for (int i = 0; i < 3; i++) {
            diwas_reflectX[i] = 2 * diwas_lineX - diwas_x[i] + margin;
        }
        g.setColor(Color.RED);
        g.drawPolygon(diwas_reflectX, diwas_y, 3);
        g.setColor(Color.RED);
        g.drawString("Reflected", diwas_reflectX[0] + 10, diwas_y[0] - 10);
    }
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int[] diwas_userX = new int[3];
        int[] diwas_userY = new int[3];

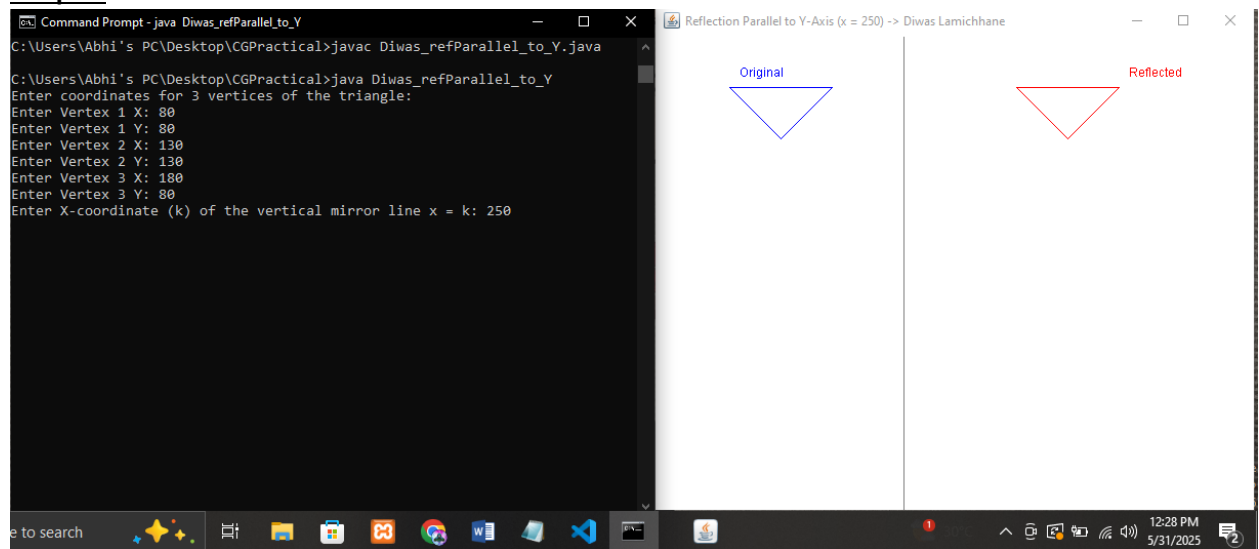
        System.out.println("Enter coordinates for 3 vertices of the triangle:");
        for (int i = 0; i < 3; i++) {
```

```

        System.out.print("Enter Vertex " + (i + 1) + " X: ");
        diwas_userX[i] = scan.nextInt();
        System.out.print("Enter Vertex " + (i + 1) + " Y: ");
        diwas_userY[i] = scan.nextInt();
    }
    System.out.print("Enter X-coordinate (k) of the vertical mirror line x = k: ");
    int diwas_k = scan.nextInt();
    scan.close();
    new Diwas_refParallel_to_Y(diwas_userX, diwas_userY, diwas_k);
}
}

```

Output:



1.13 Program to Implement 2D Reflection Through an Arbitrary Line ($y=mx+c$)

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class refY_MX_C_Kanxo extends Frame {
    int[] origX = new int[3];
    int[] origY = new int[3];
    double slope;
    double intercept;
    int width = 600, height = 600;
    public refY_MX_C_Kanxo(int[] x, int[] y, double m, double c) {
        origX = x;
        origY = y;
        slope = m;
        intercept = c;
        setTitle("Reflection through  $y = mx + c$  -> Diwas Kanxo");
        setSize(width, height);
        setVisible(true);
    }
    public void paint(Graphics g) {
        g.setColor(Color.LIGHT_GRAY);
        g.drawLine(0, height / 2, width, height / 2); // x-axis
        g.drawLine(width / 2, 0, width / 2, height); // y-axis

        g.setColor(Color.BLUE);
        int[] drawX = new int[3];
        int[] drawY = new int[3];
        for (int i = 0; i < 3; i++) {
            drawX[i] = width / 2 + origX[i];
            drawY[i] = height / 2 - origY[i];
        }
        g.drawPolygon(drawX, drawY, 3);

        int originalCentroidX = (drawX[0] + drawX[1] + drawX[2]) / 3;
        int originalCentroidY = (drawY[0] + drawY[1] + drawY[2]) / 3;
        g.setColor(Color.BLACK);
        g.drawString("Original", originalCentroidX + 10, originalCentroidY);

        int[] reflX = new int[3];
        int[] reflY = new int[3];
        for (int i = 0; i < 3; i++) {
            double x = origX[i];
            double y = origY[i];
```



```

        double d = (x + (y - intercept) * slope) / (1 + slope * slope);
        double xPrime = 2 * d - x;
        double yPrime = 2 * d * slope - y + 2 * intercept;
        reflX[i] = width / 2 + (int)Math.round(xPrime);
        reflY[i] = height / 2 - (int)Math.round(yPrime);
    }
    g.setColor(Color.RED);
    g.drawPolygon(reflX, reflY, 3);

    int reflectedCentroidX = (reflX[0] + reflX[1] + reflX[2]) / 3;
    int reflectedCentroidY = (reflY[0] + reflY[1] + reflY[2]) / 3;
    g.setColor(Color.BLACK);
    g.drawString("Reflected", reflectedCentroidX + 10, reflectedCentroidY);

    g.setColor(Color.GREEN);
    for (int x = -width / 2; x <= width / 2; x++) {
        int px = width / 2 + x;
        int py = height / 2 - (int)Math.round(slope * x + intercept);
        g.drawRect(px, py, 1, 1); // plot point
    }
    g.setColor(Color.BLACK);
    g.drawString("y = mx + c", width / 2 + 10, height / 2 - 10);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int[] kanxo_x = new int[3];
    int[] kanxo_y = new int[3];
    System.out.println("Enter coordinates of 3 triangle vertices:");
    for (int i = 0; i < 3; i++) {
        System.out.print("Enter Vertex " + (i+1) + " X: ");
        kanxo_x[i] = sc.nextInt();
        System.out.print("Enter Vertex " + (i+1) + " Y: ");
        kanxo_y[i] = sc.nextInt();
    }
    System.out.print("Enter slope (m) of line y = mx + c: ");
    double m = sc.nextDouble();
    System.out.print("Enter intercept (c) of line y = mx + c: ");
    double c = sc.nextDouble();
    sc.close();
    new refY_MX_C_Kanxo(kanxo_x, kanxo_y, m, c);
}
}

```

Output:

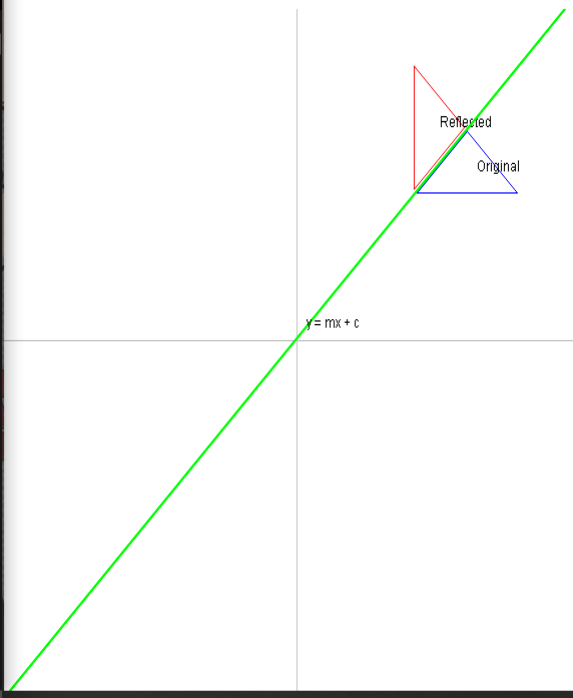
Command Prompt - java refY_MX_C_Kanxo

```
05/30/2025 02:35 PM      1,968 D_reflectionOrigin.java
05/30/2025 02:48 PM      573 fixedPointRotationAnticlkDiwas$1.class
05/30/2025 02:48 PM      3,810 fixedPointRotationAnticlkDiwas.class
05/30/2025 02:47 PM      4,838 fixedPointRotationAnticlkDiwas.java
05/30/2025 02:39 PM     515,747 ReflectionOrigin.PNG
05/30/2025 02:35 PM      70,519 ReflectionOverX.PNG
05/30/2025 02:33 PM      553 reflectionOverX_diwasdada$1.class
05/30/2025 02:33 PM      2,923 reflectionOverX_diwasdada.class
05/30/2025 02:33 PM      3,481 reflectionOverX_diwasdada.java
05/30/2025 03:13 PM      71,746 ReflectionOverY.PNG
05/30/2025 03:11 PM      553 reflectionOverY_diwasdada$1.class
05/30/2025 03:11 PM      2,926 reflectionOverY_diwasdada.class
05/30/2025 03:11 PM      3,528 reflectionOverY_diwasdada.java
05/31/2025 11:55 AM     510,250 ReflectionParallelToX.PNG
05/31/2025 12:29 PM     87,184 ReflectionParallelToY.PNG
05/31/2025 12:38 PM      3,275 refY_MX_C_Kanxo.java
05/29/2025 03:27 PM      2,787 scalingOrigin_2D.class
05/29/2025 03:26 PM      2,488 scalingOrigin_2D.java
43 File(s)      2,863,223 bytes
2 Dir(s)      7,819,038,720 bytes free

C:\Users\Abhi's PC\Desktop\CGPractical>javac refY_MX_C_Kanxo.java

C:\Users\Abhi's PC\Desktop\CGPractical>java refY_MX_C_Kanxo
Enter coordinates of 3 triangle vertices:
Enter Vertex 1 X: 120
Enter Vertex 1 Y: 120
Enter Vertex 2 X: 170
Enter Vertex 2 Y: 170
Enter Vertex 3 X: 220
Enter Vertex 3 Y: 120
Enter slope (m) of line y = mx + c: 1
Enter intercept (c) of line y = mx + c: 3
```

Reflection through $y = mx + c \rightarrow$ Diwas Kanxo



here to search  26°C Partly cloudy 12:41 PM 5/31/2025

1.14 Program to Implement 2D Shearing in the X-direction

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class ShearingX_diwas extends Frame {
    int[] putali_origX = new int[3];
    int[] putali_origY = new int[3];
    int[] putali_shearX = new int[3];
    double putali_shx;
    public ShearingX_diwas(int[] x, int[] y, double shx) {
        for (int i = 0; i < 3; i++) {
            putali_origX[i] = x[i];
            putali_origY[i] = y[i];
            putali_shearX[i] = (int) (x[i] + shx * y[i]);
        }
        this.putali_shx = shx;
        setTitle("Shearing in X-direction -> Diwas Dada");
        setSize(600, 400);
        setVisible(true);
    }
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;

        g2.translate(getWidth() / 2, getHeight() / 2);
        g2.setColor(Color.LIGHT_GRAY);
        g2.drawLine(-getWidth() / 2, 0, getWidth() / 2, 0); // X-axis
        g2.drawLine(0, -getHeight() / 2, 0, getHeight() / 2); // Y-axis

        g2.setColor(Color.BLUE);
        g2.drawPolygon(putali_origX, putali_origY, 3);
        g2.setColor(Color.BLUE);
        g2.drawString("Original", putali_origX[0] + 10, putali_origY[0] - 10);

        g2.setColor(Color.RED);
        g2.drawPolygon(putali_shearX, putali_origY, 3);
        g2.setColor(Color.RED);
        g2.drawString("Sheared", putali_shearX[0] + 10, putali_origY[0] - 10);
    }

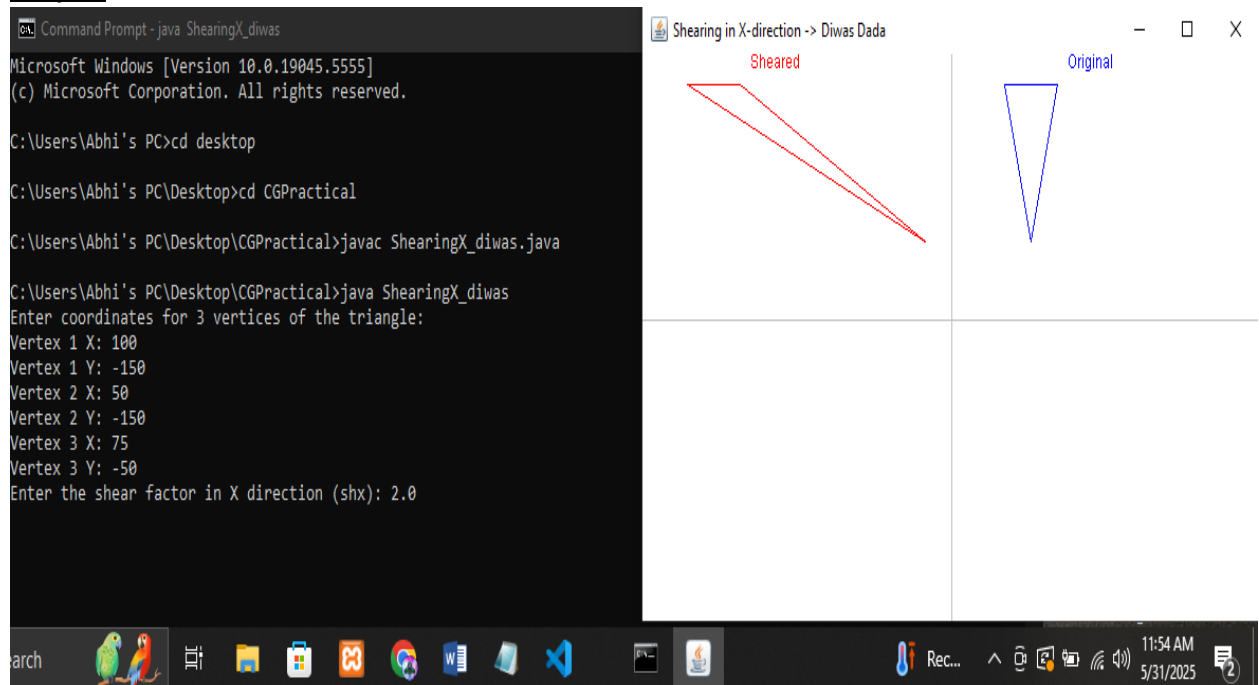
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int[] kanxi_x = new int[3];
        int[] kanxi_y = new int[3];
```

```

System.out.println("Enter coordinates for 3 vertices of the triangle:");
for (int i = 0; i < 3; i++) {
    System.out.print("Vertex " + (i + 1) + " X: ");
    kanxi_x[i] = scanner.nextInt();
    System.out.print("Vertex " + (i + 1) + " Y: ");
    kanxi_y[i] = scanner.nextInt();
}
System.out.print("Enter the shear factor in X direction (shx): ");
double kanxi_shx = scanner.nextDouble();
scanner.close();
new ShearingX_diwas(kanxi_x, kanxi_y, kanxi_shx);
}
}

```

Output:



1.15 Program to Implement 2D Shearing in the Y-direction

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class ShearingY_diwas extends Frame {
    int boss_x1, boss_y1, boss_x2, boss_y2, boss_x3, boss_y3;
    double boss_shy;
    int boss_sx1, boss_sy1, boss_sx2, boss_sy2, boss_sx3, boss_sy3;
    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;
    private final int LEGEND_OFFSET_X = 20;
    private final int LEGEND_OFFSET_Y = 40;
    private final int LEGEND_SPACING = 20;
    private final int LEGEND_BOX_SIZE = 15;
    public ShearingY_diwas() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter x1 (relative to center)");
        boss_x1 = sc.nextInt();
        System.out.println("Enter y1 (relative to center)");
        boss_y1 = sc.nextInt();
        System.out.println("Enter x2 (relative to center)");
        boss_x2 = sc.nextInt();
        System.out.println("Enter y2 (relative to center)");
        boss_y2 = sc.nextInt();
        System.out.println("Enter x3 (relative to center)");
        boss_x3 = sc.nextInt();
        System.out.println("Enter y3 (relative to center)");
        boss_y3 = sc.nextInt();
        System.out.println("Enter shearing factor in y-direction (shy)");
        boss_shy = sc.nextDouble();
        sc.close();
        boss_sx1 = boss_x1;
        boss_sy1 = (int) Math.round(boss_y1 + boss_shy * boss_x1);
        boss_sx2 = boss_x2;
        boss_sy2 = (int) Math.round(boss_y2 + boss_shy * boss_x2);
        boss_sx3 = boss_x3;
        boss_sy3 = (int) Math.round(boss_y3 + boss_shy * boss_x3);
        this.setTitle("Shearing in Y-Direction -> @Diwas");
        this.setLayout(null);
        this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
        this.setVisible(true);
        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
```

```

        System.exit(0);
    }
});
}
private Point toAWTCoords(int x, int y) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    return new Point(centerX + x, centerY - y);
}
public void paint(Graphics g) {
    int centerX = getWidth() / 2;
    int centerY = getHeight() / 2;
    g.setColor(Color.BLACK);
    g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);
    g.drawLine(0, centerY, getWidth(), centerY);
    g.drawLine(centerX, 0, centerX, getHeight());
    g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
    g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
    g.drawLine(centerX - 3, 10, centerX, 0);
    g.drawLine(centerX + 3, 10, centerX, 0);
    g.setColor(Color.BLUE);
    Point p1_orig_awt = toAWTCoords(boss_x1, boss_y1);
    Point p2_orig_awt = toAWTCoords(boss_x2, boss_y2);
    Point p3_orig_awt = toAWTCoords(boss_x3, boss_y3);
    g.drawLine(p1_orig_awt.x, p1_orig_awt.y, p2_orig_awt.x, p2_orig_awt.y);
    g.drawLine(p2_orig_awt.x, p2_orig_awt.y, p3_orig_awt.x, p3_orig_awt.y);
    g.drawLine(p3_orig_awt.x, p3_orig_awt.y, p1_orig_awt.x, p1_orig_awt.y);
    g.drawString("Original", p1_orig_awt.x, p1_orig_awt.y - 5);
    g.setColor(Color.RED);
    Point p1_sheared_awt = toAWTCoords(boss_sx1, boss_sy1);
    Point p2_sheared_awt = toAWTCoords(boss_sx2, boss_sy2);
    Point p3_sheared_awt = toAWTCoords(boss_sx3, boss_sy3);
    g.drawLine(p1_sheared_awt.x, p1_sheared_awt.y, p2_sheared_awt.x, p2_sheared_awt.y);
    g.drawLine(p2_sheared_awt.x, p2_sheared_awt.y, p3_sheared_awt.x, p3_sheared_awt.y);
    g.drawLine(p3_sheared_awt.x, p3_sheared_awt.y, p1_sheared_awt.x, p1_sheared_awt.y);
    g.drawString("Sheared", p1_sheared_awt.x, p1_sheared_awt.y - 5);
    g.setColor(Color.GREEN);
    int legendX = LEGEND_OFFSET_X;
    int legendY = LEGEND_OFFSET_Y;
    g.setColor(Color.BLUE);
    g.fillRect(legendX, legendY, LEGEND_BOX_SIZE, LEGEND_BOX_SIZE);
    g.setColor(Color.BLACK);
    g.drawString("Original", legendX + LEGEND_BOX_SIZE + 5, legendY + LEGEND_BOX_SIZE - 3);
    legendY += LEGEND_SPACING;
}

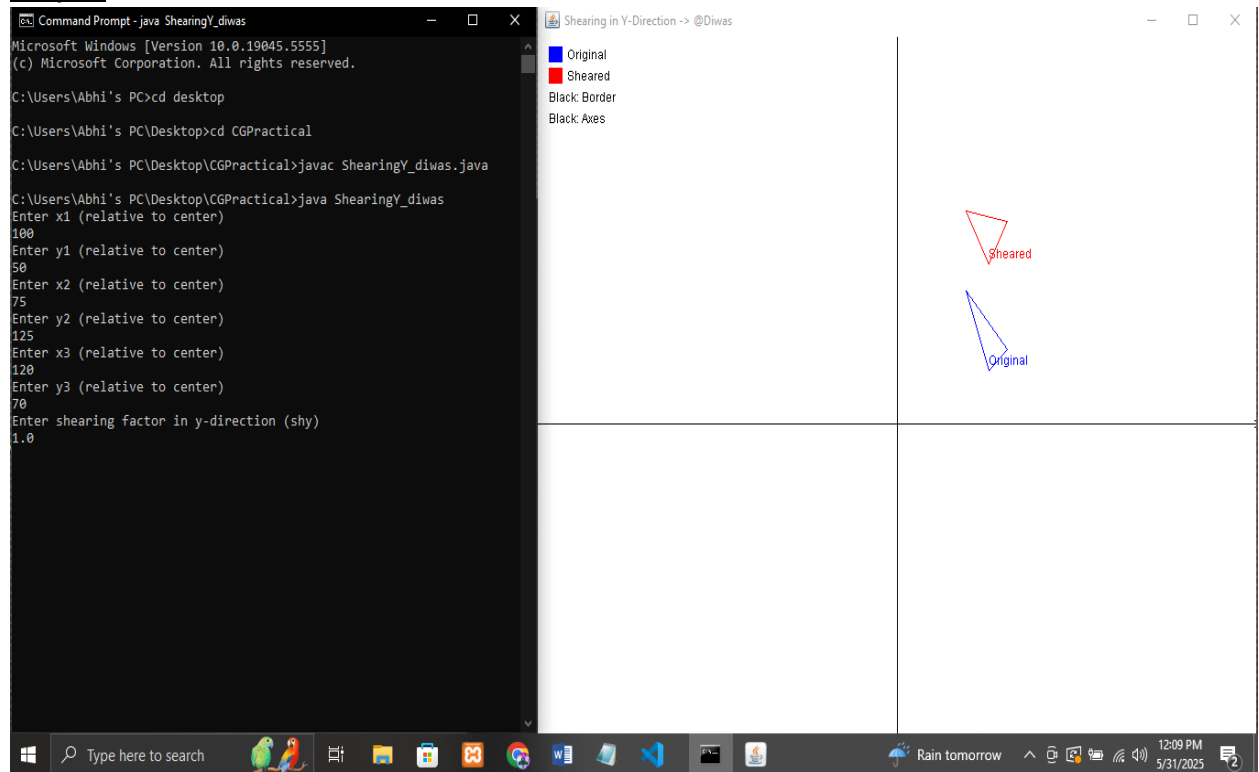
```

```

        g.setColor(Color.RED);
        g.fillRect(legendX, legendY, LEGEND_BOX_SIZE, LEGEND_BOX_SIZE);
        g.setColor(Color.BLACK);
        g.drawString("Sheared", legendX + LEGEND_BOX_SIZE + 5, legendY + LEGEND_BOX_SIZE -
3);
        legendY += LEGEND_SPACING;
        g.setColor(Color.BLACK);
        g.drawString("Black: Border", legendX, legendY + LEGEND_BOX_SIZE - 3);
        legendY += LEGEND_SPACING;
        g.setColor(Color.BLACK);
        g.drawString("Black: Axes", legendX, legendY + LEGEND_BOX_SIZE - 3);
    }
    public static void main(String[] args) {
        new ShearingY_diwas();
    }
}

```

Output:



1.16 Program to Implement 2D Shearing in the XY-direction

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class ShearingXY_diwas extends Frame {
    int[] diwas_realX = new int[4];
    int[] diwas_realY = new int[4];
    int[] diwas_shearX = new int[4];
    int[] diwas_shearY = new int[4];
    double diwas_shx, diwas_shy;

    public ShearingXY_diwas(int[] x, int[] y, double shx, double shy) {
        for (int i = 0; i < 4; i++) {
            diwas_realX[i] = x[i];
            diwas_realY[i] = y[i];
            diwas_shearX[i] = (int) (x[i] + shx * y[i]);
            diwas_shearY[i] = (int) (y[i] + shy * x[i]);
        }
        this.diwas_shx = shx;
        this.diwas_shy = shy;

        setTitle("Shearing in XY-direction -> @DIWAS");
        setSize(800, 600);
        setVisible(true);
    }
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;

        g2.translate(getWidth() / 2, getHeight() / 2);

        g2.setColor(Color.LIGHT_GRAY);
        g2.drawLine(-getWidth() / 2, 0, getWidth() / 2, 0); // X-axis
        g2.drawLine(0, -getHeight() / 2, 0, getHeight() / 2); // Y-axis

        g2.setColor(Color.BLUE);
        g2.drawPolygon(diwas_realX, diwas_realY, 4);
        g2.setColor(Color.BLUE);
        g2.drawString("Original", diwas_realX[0] + 10, diwas_realY[0] - 10);

        g2.setColor(Color.RED);
        g2.drawPolygon(diwas_shearX, diwas_shearY, 4);
        g2.setColor(Color.RED);
        g2.drawString("Sheared", diwas_shearX[0] + 10, diwas_shearY[0] - 10);
    }
}
```



```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int[] kanxo_x = new int[4];
    int[] kanxo_y = new int[4];

    System.out.println("Enter coordinates for 4 vertices of the shape:");

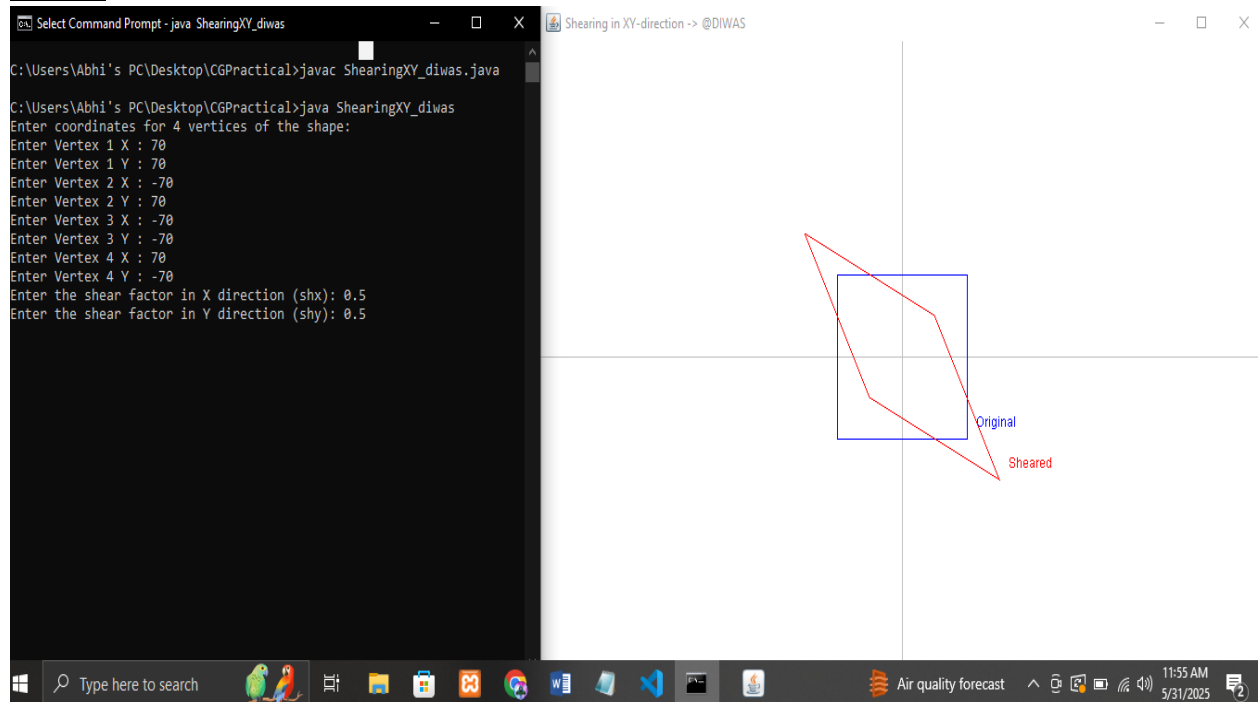
    for (int i = 0; i < 4; i++) {
        System.out.print("Enter Vertex " + (i + 1) + " X : ");
        kanxo_x[i] = Integer.parseInt(scanner.nextLine());
        System.out.print("Enter Vertex " + (i + 1) + " Y : ");
        kanxo_y[i] = Integer.parseInt(scanner.nextLine());
    }

    System.out.print("Enter the shear factor in X direction (shx): ");
    double kanxi_shx = Double.parseDouble(scanner.nextLine());
    System.out.print("Enter the shear factor in Y direction (shy): ");
    double kanxi_shy = Double.parseDouble(scanner.nextLine());
    scanner.close();

    new ShearingXY_diwas(kanxo_x, kanxo_y, kanxi_shx, kanxi_shy);
}

```

Output:



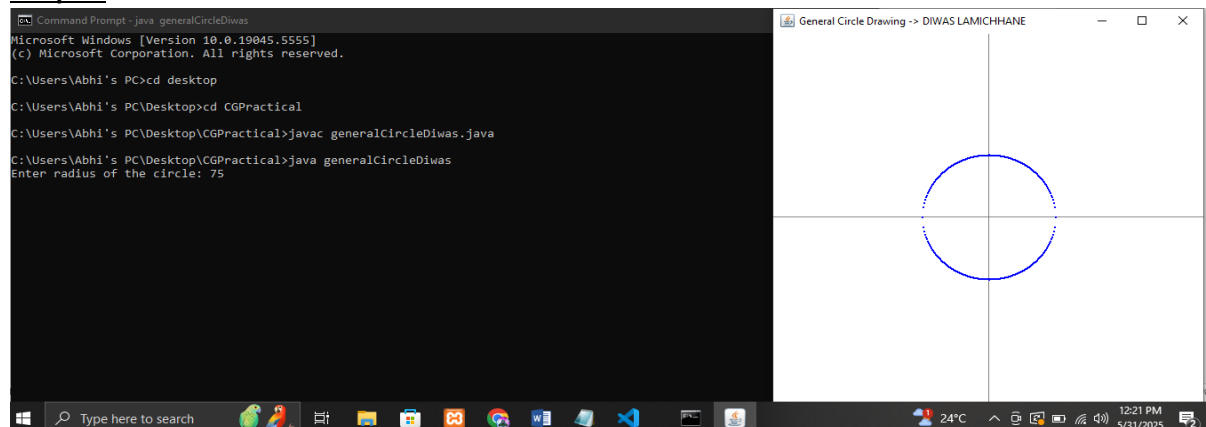
2. Circle Drawing Algorithm

2.1 Program to Implement General Circle Drawing Algorithm

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class generalCircleDiwas extends Frame {
    int radius;
    public generalCircleDiwas(int r) {
        radius = r;
        setTitle("General Circle Drawing -> DIWAS LAMICHHANE");
        setSize(500, 500);
        setVisible(true); }
    public void paint(Graphics g) {
        int xc = getWidth() / 2;
        int yc = getHeight() / 2;
        g.setColor(Color.GRAY);
        g.drawLine(0, yc, getWidth(), yc); // X-axis
        g.drawLine(xc, 0, xc, getHeight()); // Y-axis
        g.setColor(Color.BLUE);
        for (int x = -radius; x <= radius; x++) {
            double y = Math.sqrt(radius * radius - x * x);
            int xPixel = xc + x;
            int yPixel1 = yc - (int) y;
            int yPixel2 = yc + (int) y;
            g.drawRect(xPixel, yPixel1, 1, 1);
            g.drawRect(xPixel, yPixel2, 1, 1); }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter radius of the circle: ");
        int diwas_r = sc.nextInt();
        sc.close();
        new generalCircleDiwas(diwas_r); }}
```

Output:



2.2 Program to Implement Midpoint Circle Drawing Algorithm

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class midpointCircleDiwas extends Frame {
    int xc, yc, r;

    public midpointCircleDiwas(int xc, int yc, int r) {
        this.xc = xc;
        this.yc = yc;
        this.r = r;

        setTitle("Midpoint Circle Drawing Algorithm -> diwas");
        setSize(600, 400);
        setVisible(true);
    }

    public void paint(Graphics g) {
        int x = 0, y = r;
        int p = 1 - r; // Initial decision parameter

        g.setColor(Color.RED);

        drawCirclePoints(g, x, y);

        while (x < y) {
            x++;

            if (p < 0) {
                p = p + 2 * x + 1;
            } else {
                y--;
                p = p + 2 * (x - y) + 1;
            }

            drawCirclePoints(g, x, y);
        }
    }

    public void drawCirclePoints(Graphics g, int x, int y) {
        // Drawing points in all eight octants
        g.fillRect(xc + x, yc + y, 1, 1);
        g.fillRect(xc - x, yc + y, 1, 1);
        g.fillRect(xc + x, yc - y, 1, 1);
```

```

        g.fillRect(xc - x, yc - y, 1, 1);
        g.fillRect(xc + y, yc + x, 1, 1);
        g.fillRect(xc - y, yc + x, 1, 1);
        g.fillRect(xc + y, yc - x, 1, 1);
        g.fillRect(xc - y, yc - x, 1, 1);
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

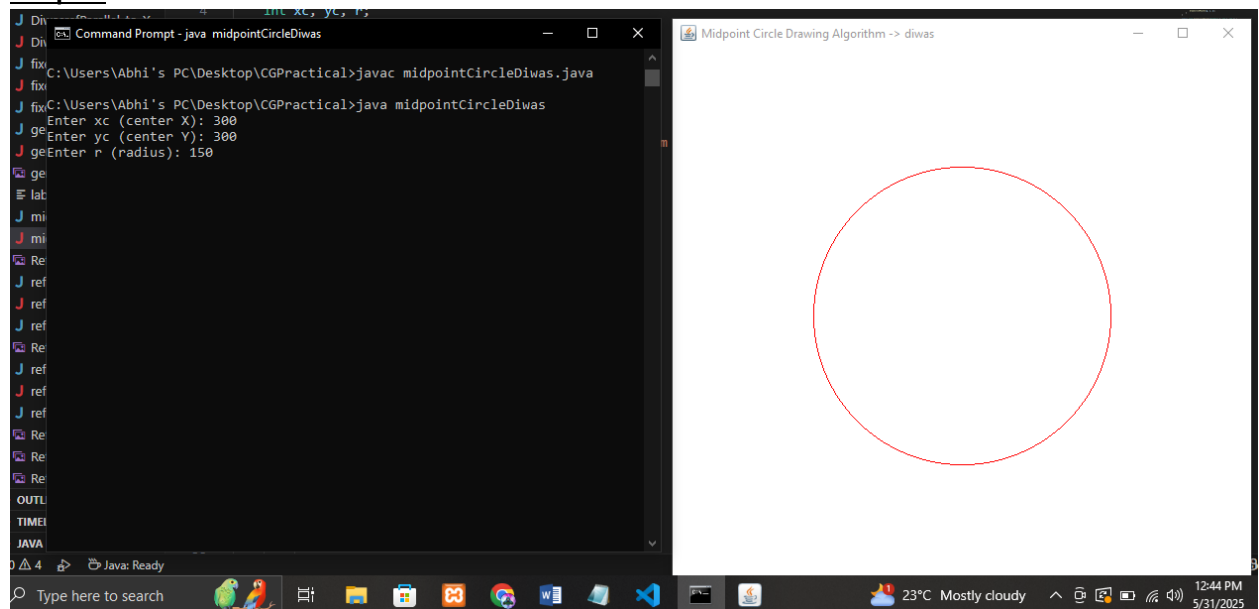
        System.out.print("Enter xc (center X): ");
        int xc = scanner.nextInt();
        System.out.print("Enter yc (center Y): ");
        int yc = scanner.nextInt();
        System.out.print("Enter r (radius): ");
        int r = scanner.nextInt();

        scanner.close();

        new midpointCircleDiwas(xc, yc, r);
    }
}

```

Output:



2.3 Program to Implement Midpoint Ellipse Drawing Algorithm

Source Code:

```
import java.util.Scanner;
import java.awt.*;
import java.awt.event.*;

class midpointEllipseDiwas extends Frame {
    int diwasCenterX, diwasCenterY, diwasRx, diwasRy;
    private final int WINDOW_WIDTH = 800;
    private final int WINDOW_HEIGHT = 800;
    private final int LEGEND_OFFSET_X = 20;
    private final int LEGEND_OFFSET_Y = 40;
    private final int LEGEND_SPACING = 20;
    private final int LEGEND_BOX_SIZE = 15;
    public midpointEllipseDiwas() {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the x-coordinate of the center :");
        diwasCenterX = sc.nextInt();
        System.out.println("Enter the y-coordinate of the center :");
        diwasCenterY = sc.nextInt();
        System.out.println("Enter the x-radius (rx):");
        diwasRx = sc.nextInt();
        System.out.println("Enter the y-radius (ry):");
        diwasRy = sc.nextInt();
        sc.close();
        this.setTitle("Midpoint Ellipse Drawing -> By Diwas Lamichhane");
        this.setLayout(null);
        this.setBounds(100, 100, WINDOW_WIDTH, WINDOW_HEIGHT);
        this.setVisible(true);
        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent we) {
                System.exit(0);
            }
        });
    }
    private Point toAWTCoords(int x, int y) {
        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2;
        return new Point(centerX + x, centerY - y);
    }
    public void paint(Graphics g) {
        int centerX = getWidth() / 2;
        int centerY = getHeight() / 2;
        g.setColor(Color.BLACK);
        g.drawRect(0, 0, getWidth() - 1, getHeight() - 1);
    }
}
```

```

g.drawLine(0, centerY, getWidth(), centerY);
g.drawLine(centerX, 0, centerX, getHeight());
g.drawLine(getWidth() - 10, centerY - 3, getWidth(), centerY);
g.drawLine(getWidth() - 10, centerY + 3, getWidth(), centerY);
g.drawLine(centerX - 3, 10, centerX, 0);
g.drawLine(centerX + 3, 10, centerX, 0);
diwasMidpointEllipse(g, diwasCenterX, diwasCenterY, diwasRx, diwasRy);
g.setColor(Color.BLUE);
Point center_awt = toAWTCords(diwasCenterX, diwasCenterY);
g.fillOval(center_awt.x - 4, center_awt.y - 4, 8, 8);
g.drawString("Center (" + diwasCenterX + ", " + diwasCenterY + ")", center_awt.x + 10,
center_awt.y);
int legendX = LEGEND_OFFSET_X;
int legendY = LEGEND_OFFSET_Y;
g.setColor(Color.BLUE);
g.fillRect(legendX, legendY, LEGEND_BOX_SIZE, LEGEND_BOX_SIZE);
g.setColor(Color.BLACK);
g.drawString("Ellipse Center", legendX + LEGEND_BOX_SIZE + 5, legendY +
LEGEND_BOX_SIZE - 3);
legendY += LEGEND_SPACING;
g.setColor(Color.RED);
g.fillRect(legendX, legendY, LEGEND_BOX_SIZE, LEGEND_BOX_SIZE);
g.setColor(Color.BLACK);
g.drawString("Drawn Ellipse", legendX + LEGEND_BOX_SIZE + 5, legendY +
LEGEND_BOX_SIZE - 3);
legendY += LEGEND_SPACING;
g.setColor(Color.BLACK);
g.drawString("Black: Border/Axes", legendX, legendY + LEGEND_BOX_SIZE - 3);
}

private void diwasMidpointEllipse(Graphics diwasg, int diwasPlotCenterX_std, int
diwasPlotCenterY_std, int diwasPlotRx, int diwasPlotRy) {
    double diwasdx, diwasdy, diwasd1, diwasd2;
    int diwasx_offset, diwasy_offset;
    diwasx_offset = 0;
    diwasy_offset = diwasPlotRy;
    diwasd1 = diwasPlotRy * diwasPlotRy - diwasPlotRx * diwasPlotRx * diwasPlotRy + 0.25 *
diwasPlotRx * diwasPlotRx;
    diwasdx = 2 * diwasPlotRy * diwasPlotRy * diwasx_offset;
    diwasdy = 2 * diwasPlotRx * diwasPlotRx * diwasy_offset;
    diwasEllipsePoints(diwasg, diwasPlotCenterX_std, diwasPlotCenterY_std, diwasx_offset,
diwasy_offset);
    while (diwasdx < diwasdy) {
        if (diwasd1 < 0) {
            diwasx_offset++;

```

```

        diwasdx = diwasdx + 2 * diwasPlotRy * diwasPlotRy;
        diwasd1 = diwasd1 + diwasdx + diwasPlotRy * diwasPlotRy;
    } else {
        diwasx_offset++;
        diwasy_offset--;
        diwasdx = diwasdx + 2 * diwasPlotRy * diwasPlotRy;
        diwasdy = diwasdy - 2 * diwasPlotRx * diwasPlotRx;
        diwasd1 = diwasd1 + diwasdx - diwasdy + diwasPlotRy * diwasPlotRy;
    }
    diwasEllipsePoints(diwasg, diwasPlotCenterX_std, diwasPlotCenterY_std, diwasx_offset,
diwasy_offset);
    }
    diwasd2 = diwasPlotRy * diwasPlotRy * (diwasx_offset + 0.5) * (diwasx_offset + 0.5) +
diwasPlotRx * diwasPlotRx * (diwasy_offset - 1) * (diwasy_offset - 1) - diwasPlotRx *
diwasPlotRx * diwasPlotRy * diwasPlotRy;
    while (diwasy_offset >= 0) {
        diwasEllipsePoints(diwasg, diwasPlotCenterX_std, diwasPlotCenterY_std, diwasx_offset,
diwasy_offset);
        if (diwasd2 > 0) {
            diwasy_offset--;
            diwasdy = diwasdy - 2 * diwasPlotRx * diwasPlotRx;
            diwasd2 = diwasd2 + diwasPlotRx * diwasPlotRx - diwasdy;
        } else {
            diwasy_offset--;
            diwasx_offset++;
            diwasdx = diwasdx + 2 * diwasPlotRy * diwasPlotRy;
            diwasdy = diwasdy - 2 * diwasPlotRx * diwasPlotRx;
            diwasd2 = diwasd2 + diwasdx - diwasdy + diwasPlotRx * diwasPlotRx;
        }
    }
}

private void diwasEllipsePoints(Graphics diwasg, int diwasPlotCenterX_std, int
diwasPlotCenterY_std, int diwasPlotX_offset, int diwasPlotY_offset) {
    diwasg.setColor(Color.RED);
    Point p_awt;
    p_awt = toAWTCords(diwasPlotCenterX_std + diwasPlotX_offset, diwasPlotCenterY_std +
diwasPlotY_offset);
    diwasg.fillRect(p_awt.x, p_awt.y, 1, 1);
    p_awt = toAWTCords(diwasPlotCenterX_std - diwasPlotX_offset, diwasPlotCenterY_std +
diwasPlotY_offset);
    diwasg.fillRect(p_awt.x, p_awt.y, 1, 1);
    p_awt = toAWTCords(diwasPlotCenterX_std + diwasPlotX_offset, diwasPlotCenterY_std -
diwasPlotY_offset);
    diwasg.fillRect(p_awt.x, p_awt.y, 1, 1);

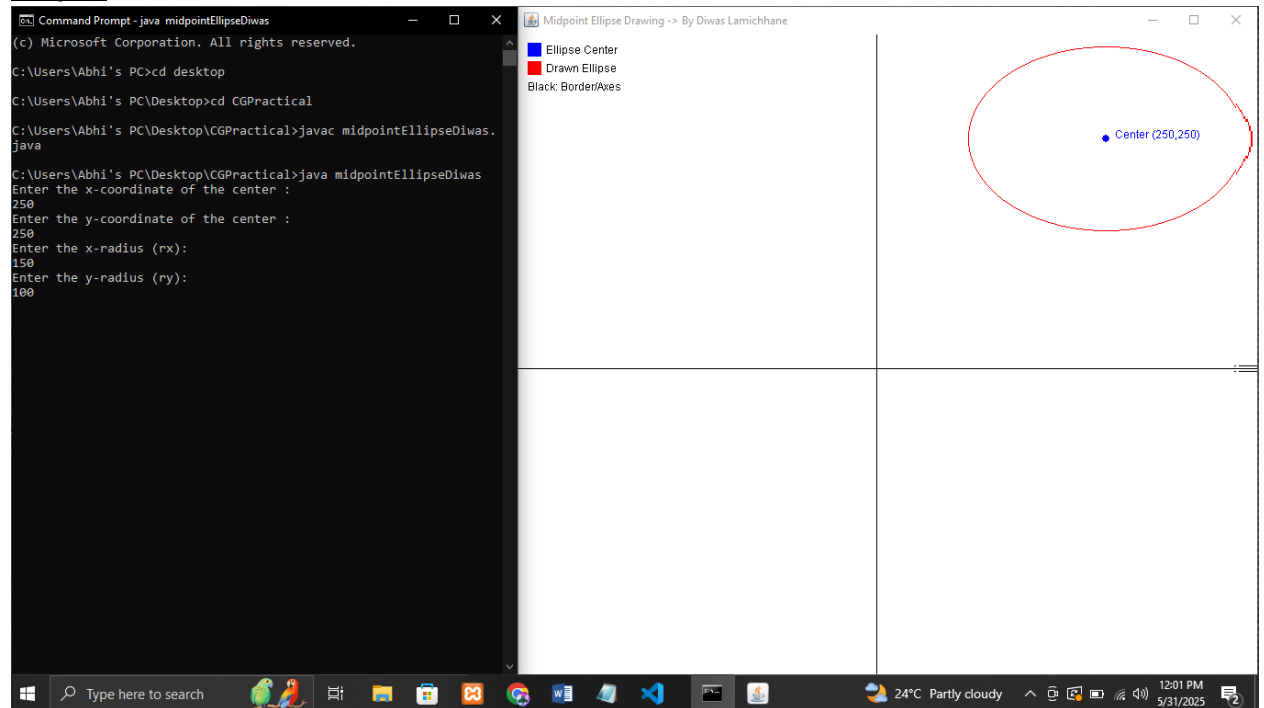
```

```

        p_awt = toAWTCoords(diwasPlotCenterX_std - diwasPlotX_offset, diwasPlotCenterY_std -
diwasPlotY_offset);
        diwasg.fillRect(p_awt.x, p_awt.y, 1, 1);
    }
    public static void main(String[] args) {
        new midpointEllipseDiwas();
    }
}

```

Output:



3. Line Drawing Algorithm

3.1 Program to Implement DDA Line Drawing Algorithm

Source Code:

```
import java.awt.*;
import java.util.Scanner;

public class DDA_diwas extends Frame {
    int x1, y1, x2, y2;

    public DDA_diwas(int x1, int y1, int x2, int y2) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
        setTitle("DDA Line Drawing -> Diwas");
        setSize(600, 400);
        setVisible(true);
    }

    public void paint(Graphics g) {
        int width = getWidth();
        int height = getHeight();
        int centerX = width / 2;
        int centerY = height / 2;

        // Draw X-axis
        g.setColor(Color.BLACK);
        g.drawLine(0, centerY, width, centerY);

        // Draw Y-axis
        g.drawLine(centerX, 0, centerX, height);

        // Adjust coordinates to center-origin system
        int dx = x2 - x1;
        int dy = y2 - y1;
        int steps = Math.abs(dx) > Math.abs(dy) ? Math.abs(dx) : Math.abs(dy);
        float Xinc = (float) dx / steps;
        float Yinc = (float) dy / steps;
        float x = x1;
        float y = y1;

        g.setColor(Color.RED);
        for (int i = 0; i <= steps; i++) {
            int px = Math.round(x) + centerX; // shift origin to center
```

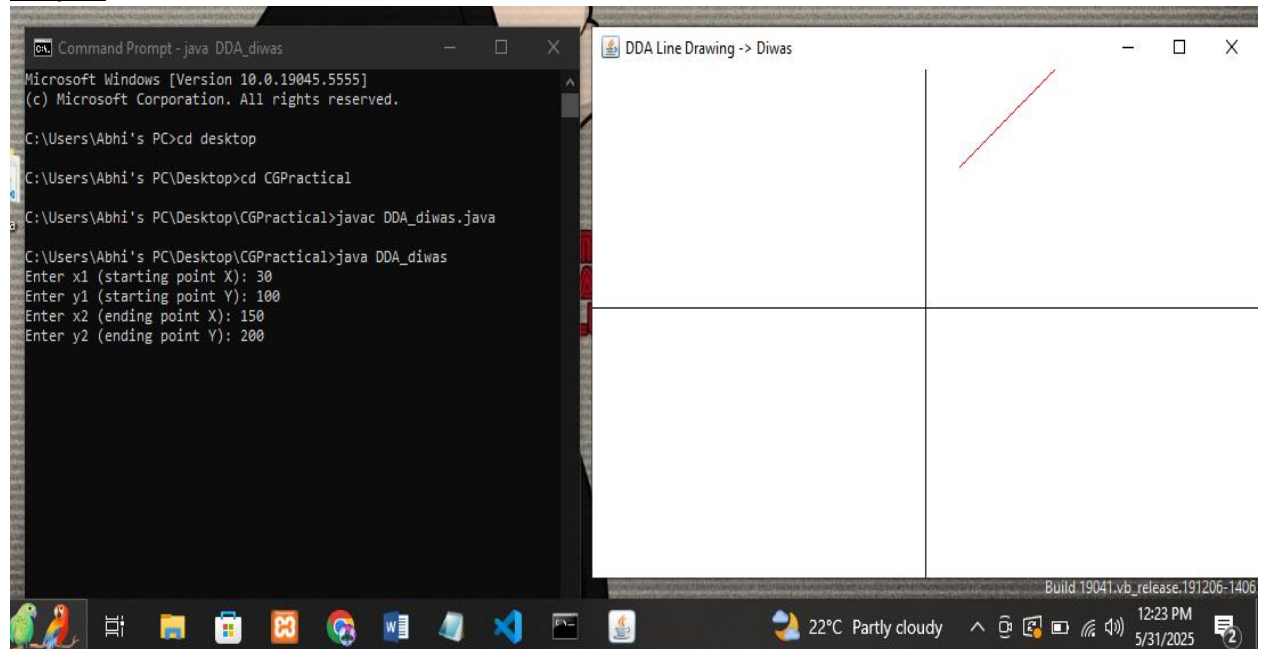
```

        int py = centerY - Math.round(y); // invert Y to match standard Cartesian
        g.fillRect(px, py, 1, 1);
        x += Xinc;
        y += Yinc;
    }
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter x1 (starting point X): ");
    int x1 = scanner.nextInt();
    System.out.print("Enter y1 (starting point Y): ");
    int y1 = scanner.nextInt();
    System.out.print("Enter x2 (ending point X): ");
    int x2 = scanner.nextInt();
    System.out.print("Enter y2 (ending point Y): ");
    int y2 = scanner.nextInt();
    scanner.close();
    new DDA_diwas(x1, y1, x2, y2);
}
}

```

Output:



3.2 Program to Implement Bresenham's Line Drawing Algorithm (BLA)

Source Code:

```
import java.awt.*;
import java.util.Scanner;
public class BLA_diwas extends Frame {
    int x1, y1, x2, y2;
    public BLA_diwas(int x1, int y1, int x2, int y2) {
        this.x1 = x1;
        this.y1 = y1;
        this.x2 = x2;
        this.y2 = y2;
        setTitle("Bresenham's Line Drawing Algorithm (BLA) - DIWAS LAMICHHANE");
        setSize(600, 400);
        setVisible(true); }
    public void paint(Graphics g) {
        int width = getWidth();
        int height = getHeight();
        int centerX = width / 2;
        int centerY = height / 2;
        g.setColor(Color.BLACK);
        g.drawLine(0, centerY, width, centerY); // X-axis
        g.drawLine(centerX, 0, centerX, height); // Y-axis
        int xStart = x1;
        int yStart = y1;
        int xEnd = x2;
        int yEnd = y2;
        int dx = xEnd - xStart;
        int dy = yEnd - yStart;
        int absDx = Math.abs(dx);
        int absDy = Math.abs(dy);
        int x = xStart;
        int y = yStart;
        g.setColor(Color.RED);
        if (absDy <= absDx) { // Line is more horizontal
            int p = 2 * absDy - absDx;
            int stepX = dx > 0 ? 1 : -1;
            int stepY = dy > 0 ? 1 : -1;
            for (int i = 0; i <= absDx; i++) {
                int px = x + centerX;
                int py = centerY - y;
                g.fillRect(px, py, 1, 1);
                x += stepX;
                if (p < 0) {
                    p += 2 * absDy;
                }
            }
        }
    }
}
```

```

    } else {
        y += stepY;
        p += 2 * (absDy - absDx);
    }} else { // Line is more vertical
int p = 2 * absDx - absDy;
int stepX = dx > 0 ? 1 : -1;
int stepY = dy > 0 ? 1 : -1;
for (int i = 0; i <= absDy; i++) {
    int px = x + centerX;
    int py = centerY - y;
    g.fillRect(px, py, 1, 1);
    y += stepY;
    if (p < 0) {
        p += 2 * absDx;
    } else {
        x += stepX;
        p += 2 * (absDx - absDy);
    }}}
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter x1 (starting point X): ");
    int x1 = scanner.nextInt();
    System.out.print("Enter y1 (starting point Y): ");
    int y1 = scanner.nextInt();
    System.out.print("Enter x2 (ending point X): ");
    int x2 = scanner.nextInt();
    System.out.print("Enter y2 (ending point Y): ");
    int y2 = scanner.nextInt();
    scanner.close();
    new BLA_diwas(x1, y1, x2, y2);
}
}

```

Output:

The screenshot shows a Windows desktop environment. On the left, a Command Prompt window titled "Command Prompt - java BLA_diwas" displays the execution of the program. It shows the compilation of several Java files and the execution of the main method. The user inputs the following values: x1 (starting point X): 30, y1 (starting point Y): 30, x2 (ending point X): 200, and y2 (ending point Y): 200. On the right, a window titled "Bresenham's Line Drawing Algorithm (BLA) - DIWAS LAMICHANE" displays a 2D coordinate system with a red line drawn from the point (30, 30) to (200, 200). The line is plotted on a grid. The Windows taskbar at the bottom shows the time as 12:42 PM on 5/31/2025, and the system temperature as 22°C Mostly clear.

4. Area Filling Algorithm

4.1 Program to Implement Boundary Fill Algorithm

Source Code:

```
import java.awt.*;
import java.awt.event.*;

public class DiwasBoundaryFill extends Frame {
    private int[][] image;
    private final int WIDTH = 500, HEIGHT = 500;
    private final int COLOR_FILL = 3;
    private final int COLOR_BOUNDARY = 1;
    private final int COLOR_EMPTY = 2;

    public DiwasBoundaryFill() {
        setTitle("Boundary Fill Algorithm");
        setSize(WIDTH, HEIGHT);
        setVisible(true);
        setLocation(100, 100);

        image = new int[WIDTH][HEIGHT];
        for (int i = 0; i < WIDTH; i++) {
            for (int j = 0; j < HEIGHT; j++) {
                if (i < 100 || i > 400 || j < 100 || j > 400) {
                    image[i][j] = COLOR_BOUNDARY;
                } else {
                    image[i][j] = COLOR_EMPTY;
                }
            }
        }

        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                int x = e.getX();
                int y = e.getY();
                if (image[x][y] != COLOR_BOUNDARY && image[x][y] != COLOR_FILL) {
                    boundaryFill(x, y, image[x][y], COLOR_FILL);
                    repaint();
                }
            }
        });
    }

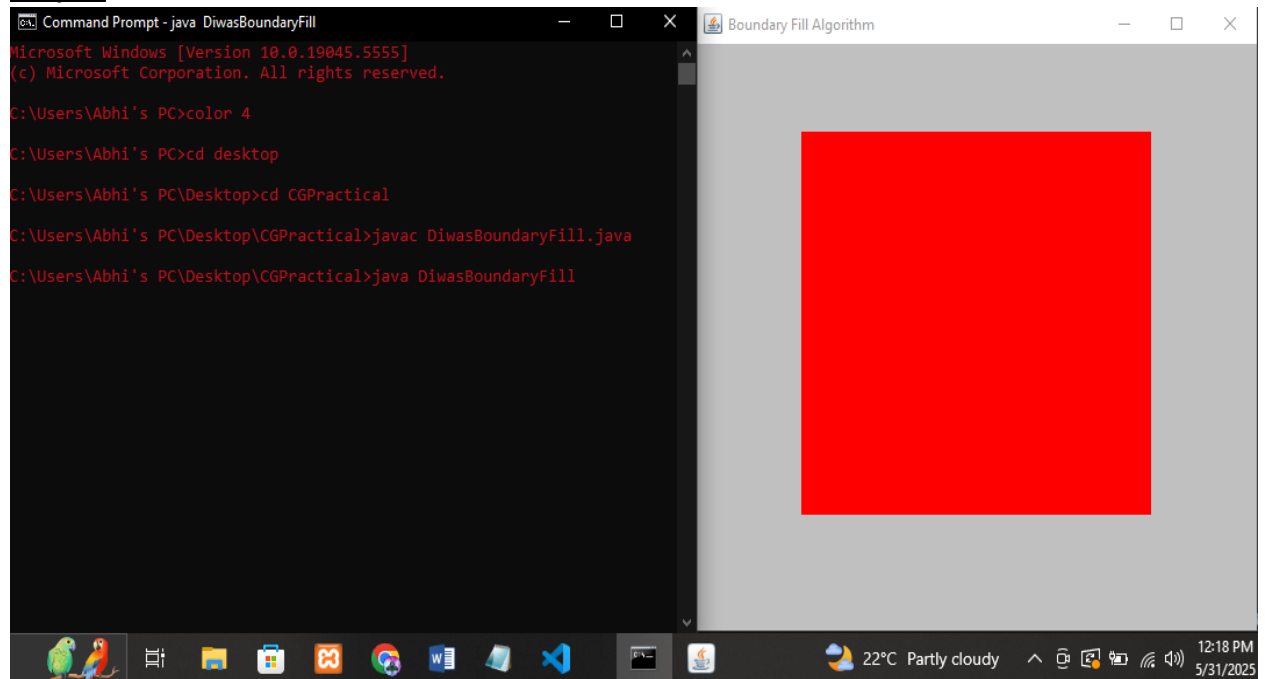
    public void boundaryFill(int x, int y, int targetColor, int replacementColor) {
        if (x < 0 || x >= WIDTH || y < 0 || y >= HEIGHT || image[x][y] == targetColor || image[x][y]
        == COLOR_BOUNDARY) {
            return;
        }
        image[x][y] = replacementColor;
```

```

        boundaryFill(x + 1, y, targetColor, replacementColor); // Right
        boundaryFill(x - 1, y, targetColor, replacementColor); // Left
        boundaryFill(x, y + 1, targetColor, replacementColor); // Down
        boundaryFill(x, y - 1, targetColor, replacementColor); // Up
    }
    @Override
    public void paint(Graphics g) {
        for (int i = 0; i < WIDTH; i++) {
            for (int j = 0; j < HEIGHT; j++) {
                switch (image[i][j]) {
                    case 1: g.setColor(Color.LIGHT_GRAY); break;
                    case 2: g.setColor(Color.RED); break;
                    case 3: g.setColor(Color.BLUE); break;
                }
                g.fillRect(i, j, 1, 1);
            }
        }
    }
    public static void main(String[] args) {
        new DiwasBoundaryFill();
    }
}

```

Output:



4.2 Program to Implement Flood Fill Algorithm

Source Code:

```
import java.awt.*;
import java.awt.event.*;

public class floodFill_diwas extends Frame {
    private int[][] image;
    private final int WIDTH = 500, HEIGHT = 500;
    private final int COLOR_FILL = 3;
    private final int COLOR_EMPTY = 1;
    public floodFill_diwas() {
        setTitle("Flood Fill Algorithm");
        setSize(WIDTH, HEIGHT);
        setVisible(true);
        setLocation(100, 100);
        image = new int[WIDTH][HEIGHT];
        for (int i = 0; i < WIDTH; i++) {
            for (int j = 0; j < HEIGHT; j++) {
                if (i % 2 == 0 && j % 2 == 0) {
                    image[i][j] = COLOR_EMPTY;
                } else {
                    image[i][j] = 2; // Use 2 for an initial color
                }
            }
        }
        addMouseListener(new MouseAdapter() {
            @Override
            public void mousePressed(MouseEvent e) {
                int x = e.getX();
                int y = e.getY();
                int targetColor = image[x][y];
                if (targetColor != COLOR_FILL) { // Avoid flood fill in already filled area
                    floodFill(x, y, targetColor, COLOR_FILL);
                    repaint(); // Repaint the canvas to reflect the change
                }
            }
        });
        public void floodFill(int x, int y, int targetColor, int replacementColor) {
            if (x < 0 || x >= WIDTH || y < 0 || y >= HEIGHT || image[x][y] != targetColor || image[x][y]
            == replacementColor) {
                return;
            }
            image[x][y] = replacementColor;

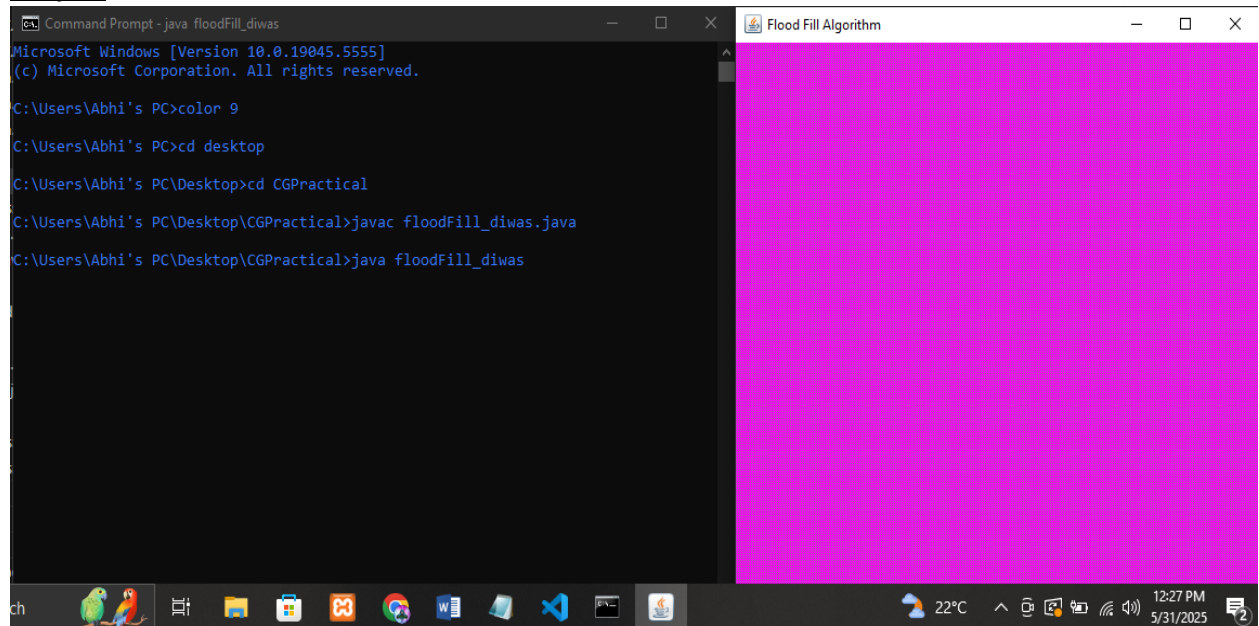
            floodFill(x + 1, y, targetColor, replacementColor); // Down
```

```

        floodFill(x - 1, y, targetColor, replacementColor); // Up
        floodFill(x, y + 1, targetColor, replacementColor); // Right
        floodFill(x, y - 1, targetColor, replacementColor); // Left
    }
    @Override
    public void paint(Graphics g) {
        for (int i = 0; i < WIDTH; i++) {
            for (int j = 0; j < HEIGHT; j++) {
                switch (image[i][j]) {
                    case 1: g.setColor(Color.gray); break; // Empty color
                    case 2: g.setColor(Color.magenta); break; // Initial color
                    case 3: g.setColor(Color.blue); break; // Fill color
                }
                g.fillRect(i, j, 1, 1); // Draw a 1x1 rectangle at (i, j)
            }
        }
    }
    public static void main(String[] args) {
        new floodFill_diwas();
    }
}

```

Output:



5. Clipping

5.1 Program to Implement Line Clipping

Source Code:

```
package clipping;
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*;
import java.util.Scanner;

public class clippingDiwas extends JPanel {
    private int deboss_xmin, deboss_ymin, deboss_xmax, deboss_ymax;
    private int deewosh_x1, deewosh_y1, deewosh_x2, deewosh_y2;
    private int diwas_cx1, diwas_cy1, diwas_cx2, diwas_cy2;
    private boolean lamichhane_clipped = false;
    private boolean diwas_showClipped = false;
    public clippingDiwas(int xmin, int ymin, int xmax, int ymax, int x1, int y1, int x2, int y2) {
        this.deboss_xmin = xmin;
        this.deboss_ymin = ymin;
        this.deboss_xmax = xmax;
        this.deboss_ymax = ymax;
        this.deewosh_x1 = x1;
        this.deewosh_y1 = y1;
        this.deewosh_x2 = x2;
        this.deewosh_y2 = y2;
        lamichhane_clipped = lamichhane_cohenSutherlandClip();
        addMouseListener(new MouseAdapter() {
            @Override
            public void mouseClicked(MouseEvent e) {
                diwas_showClipped = !diwas_showClipped;
                repaint();
            }
        });
    }

    private static final int INSIDE = 0;
    private static final int LEFT = 1;
    private static final int RIGHT = 2;
    private static final int BOTTOM = 4;
    private static final int TOP = 8;
    private int kanxo_computeOutCode(int x, int y) {
        int code = INSIDE;
        if (x < deboss_xmin) code |= LEFT;
        else if (x > deboss_xmax) code |= RIGHT;
        if (y < deboss_ymin) code |= BOTTOM;
        else if (y > deboss_ymax) code |= TOP;
        return code;
    }
}
```

```

}
private boolean lamichhane_cohenSutherlandClip() {
    int x0 = deewosh_x1, y0 = deewosh_y1, x1_ = deewosh_x2, y1__ = deewosh_y2;
    int outcode0 = kanxo_computeOutCode(x0, y0);
    int outcode1 = kanxo_computeOutCode(x1_, y1__);
    boolean accept = false;
    while (true) {
        if ((outcode0 | outcode1) == 0) {
            accept = true;
            break;
        } else if ((outcode0 & outcode1) != 0) {
            break;
        } else {
            int outcodeOut = (outcode0 != 0) ? outcode0 : outcode1;
            int x = 0, y = 0;
            if ((outcodeOut & TOP) != 0) {
                x = x0 + (x1_ - x0) * (deboss_ymax - y0) / (y1__ - y0);
                y = deboss_ymax;
            } else if ((outcodeOut & BOTTOM) != 0) {
                x = x0 + (x1_ - x0) * (deboss_ymin - y0) / (y1__ - y0);
                y = deboss_ymin;
            } else if ((outcodeOut & RIGHT) != 0) {
                y = y0 + (y1__ - y0) * (deboss_xmax - x0) / (x1_ - x0);
                x = deboss_xmax;
            } else if ((outcodeOut & LEFT) != 0) {
                y = y0 + (y1__ - y0) * (deboss_xmin - x0) / (x1_ - x0);
                x = deboss_xmin;
            }
        }
        if (outcodeOut == outcode0) {
            x0 = x;
            y0 = y;
            outcode0 = kanxo_computeOutCode(x0, y0);
        } else {
            x1_ = x;
            y1__ = y;
            outcode1 = kanxo_computeOutCode(x1_, y1__);
        }
    }
}
if (accept) {
    diwas_cx1 = x0;
    diwas_cy1 = y0;
    diwas_cx2 = x1_;
    diwas_cy2 = y1__;
}

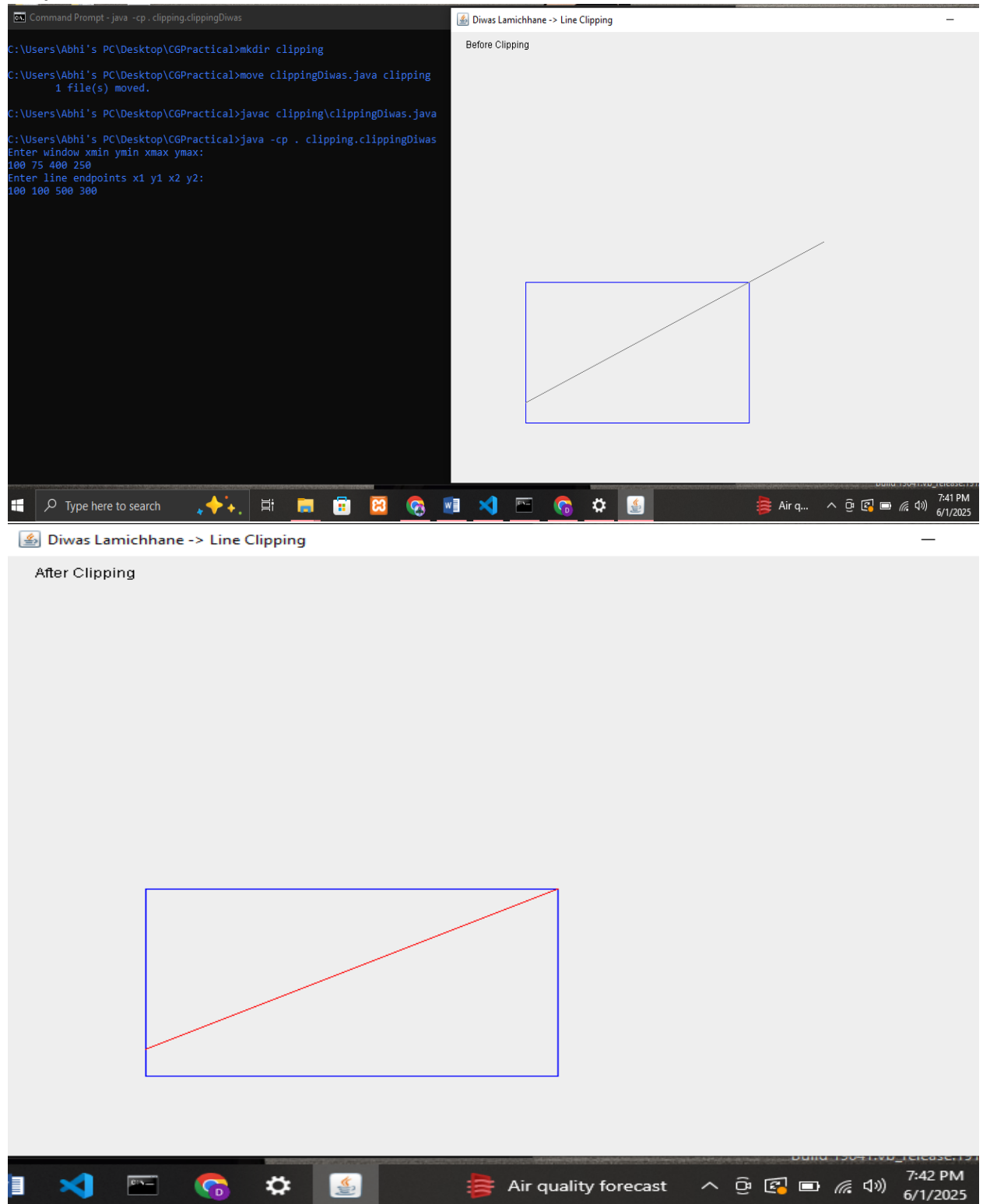
```

```

        return true;
    }
    return false;
}
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.setColor(Color.BLUE);
    g.drawRect(deboss_xmin, getHeight() - deboss_ymax, deboss_xmax - deboss_xmin,
deboss_ymax - deboss_ymin);
    if (!diwas_showClipped) {
        g.setColor(Color.GRAY);
        g.drawLine(deewosh_x1, getHeight() - deewosh_y1, deewosh_x2, getHeight() -
deewosh_y2);
        g.setColor(Color.BLACK);
        g.drawString("Before Clipping", 20, 20);
    } else {
        if (lamichhane_clipped) {
            g.setColor(Color.RED);
            g.drawLine(diwas_cx1, getHeight() - diwas_cy1, diwas_cx2, getHeight() - diwas_cy2);
            g.setColor(Color.BLACK);
            g.drawString("After Clipping ", 20, 20);
        } else {
            g.setColor(Color.RED);
            g.drawString("Line is outside the window ", 20, 20);}}}
    public static void main(String[] args) {
        Scanner diwas_scanner = new Scanner(System.in);
        System.out.println("Enter window xmin ymin xmax ymax:");
        int deboss_xmin = diwas_scanner.nextInt();
        int deboss_ymin = diwas_scanner.nextInt();
        int deboss_xmax = diwas_scanner.nextInt();
        int deboss_ymax = diwas_scanner.nextInt();
        System.out.println("Enter line endpoints x1 y1 x2 y2:");
        int deewosh_x1 = diwas_scanner.nextInt();
        int deewosh_y1 = diwas_scanner.nextInt();
        int deewosh_x2 = diwas_scanner.nextInt();
        int deewosh_y2 = diwas_scanner.nextInt();
        JFrame shaiHope_frame = new JFrame("Diwas Lamichhane -> Line Clipping");
        shaiHope_frame.add(new clippingDiwas(deboss_xmin, deboss_ymin, deboss_xmax,
deboss_ymax, deewosh_x1, deewosh_y1, deewosh_x2, deewosh_y2));
        shaiHope_frame.setSize(800, 600);
        shaiHope_frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        shaiHope_frame.setVisible(true);
    }
}

```

Output:



6. 3D Transformation

6.1 Program to Implement 3D Translation

Source Code:

```
import javax.swing.*.*;
import java.awt.*.*;
import java.util.Scanner;

public class ThreeDTranslation_lamichhane extends JPanel {
    private int[][] dl_cube1 = new int[8][2];
    private int[][] dl_cube2 = new int[8][2];
    public ThreeDTranslation_lamichhane(int[][] as_vertices, int ab_tx, int ab_ty) {
        for (int i = 0; i < 8; i++) {
            dl_cube1[i] = as_vertices[i];
            dl_cube2[i][0] = as_vertices[i][0] + ab_tx;
            dl_cube2[i][1] = as_vertices[i][1] + ab_ty;
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int w = getWidth();
        int h = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, h / 2, w, h / 2);
        g.drawLine(w / 2, 0, w / 2, h);
        int xOffset = w / 2;
        int yOffset = h / 2;
        as_drawCube(g, dl_cube1, Color.BLACK, "Original", "Cube 1", xOffset, yOffset);
        as_drawCube(g, dl_cube2, new Color(150, 75, 0), "Translated", "Cube 2", xOffset, yOffset);
    }
    private void as_drawCube(Graphics g, int[][] ba_cube, Color as_edgeColor, String ab_label,
        String ab_name, int xOffset, int yOffset) {
        g.setColor(as_edgeColor);
        as_drawLine(g, ba_cube, 0, 1, xOffset, yOffset);
        as_drawLine(g, ba_cube, 1, 2, xOffset, yOffset);
        as_drawLine(g, ba_cube, 2, 3, xOffset, yOffset);
        as_drawLine(g, ba_cube, 3, 0, xOffset, yOffset);
        as_drawLine(g, ba_cube, 4, 5, xOffset, yOffset);
        as_drawLine(g, ba_cube, 5, 6, xOffset, yOffset);
        as_drawLine(g, ba_cube, 6, 7, xOffset, yOffset);
        as_drawLine(g, ba_cube, 7, 4, xOffset, yOffset);
        for (int i = 0; i < 4; i++) {
            as_drawLine(g, ba_cube, i, i + 4, xOffset, yOffset);
        }
    }
}
```

```

    for (int i = 0; i < 8; i++) {
        g.fillOval(ba_cube[i][0] + xOffset - 4, flipY(ba_cube[i][1]) - 4, 8, 8);
    }
    g.setFont(new Font("Arial", Font.PLAIN, 14));
    g.drawString(ab_label, ba_cube[0][0] + xOffset - 20, flipY(ba_cube[0][1]) - 20);
    g.setFont(new Font("Arial", Font.BOLD, 16));
}

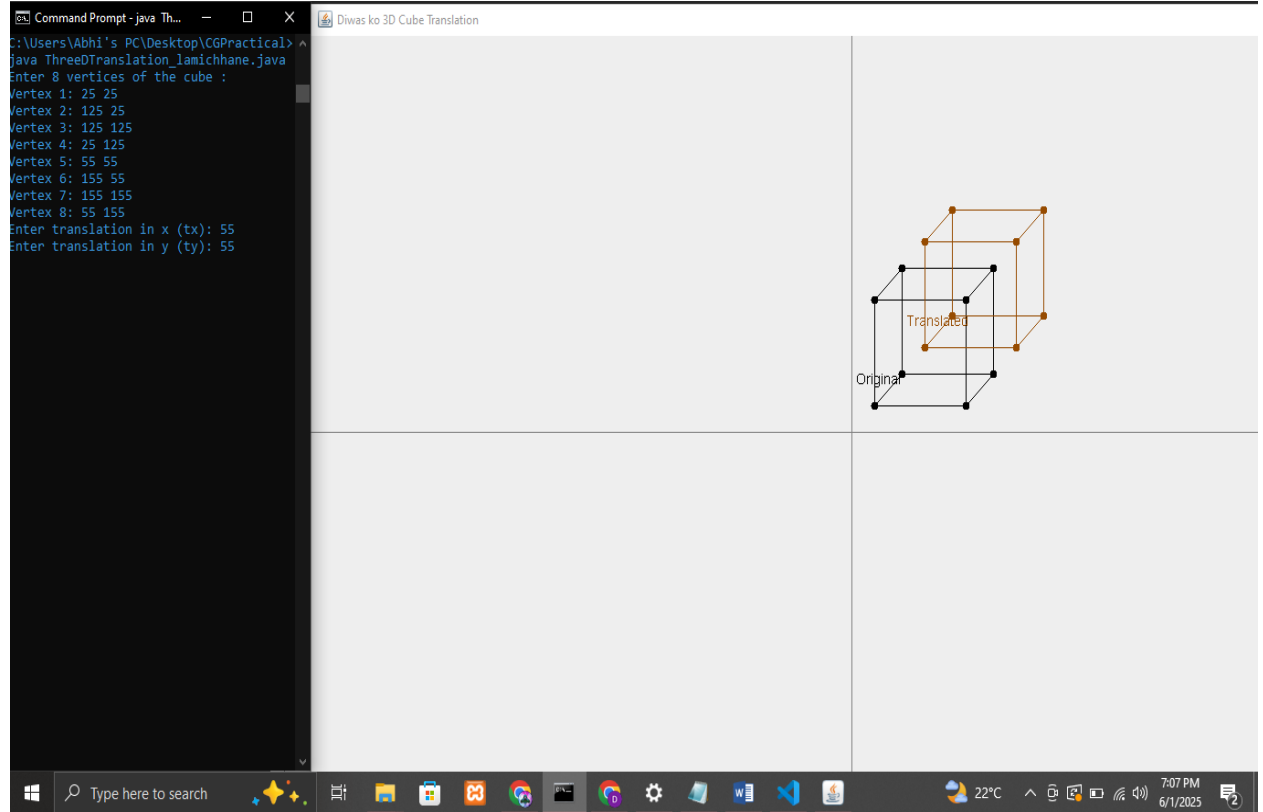
private void as_drawLine(Graphics g, int[][] ba_cube, int diw_i, int diw_j, int xOffset, int
yOffset) {
    g.drawLine(ba_cube[diw_i][0] + xOffset, flipY(ba_cube[diw_i][1]), ba_cube[diw_j][0] + xOffset,
flipY(ba_cube[diw_j][1]));
}

private int flipY(int y) {
    return getHeight() / 2 - y;
}

public static void main(String[] args) {
    Scanner lamichhane_scanner = new Scanner(System.in);
    int[][] sigdel_vertices = new int[8][2];
    System.out.println("Enter 8 vertices of the cube :");
    for (int i = 0; i < 8; i++) {
        System.out.print("Vertex " + (i + 1) + ": ");
        sigdel_vertices[i][0] = lamichhane_scanner.nextInt();
        sigdel_vertices[i][1] = lamichhane_scanner.nextInt();
    }
    System.out.print("Enter translation in x (tx): ");
    int diwas_tx = lamichhane_scanner.nextInt();
    System.out.print("Enter translation in y (ty): ");
    int diwas_ty = lamichhane_scanner.nextInt();
    JFrame lamichhane_frame = new JFrame("Diwas ko 3D Cube Translation");
    ThreeDTranslation_lamichhane lamichhane_panel = new
    ThreeDTranslation_lamichhane(sigdel_vertices, diwas_tx, diwas_ty);
    lamichhane_frame.add(lamichhane_panel);
    lamichhane_frame.setSize(1200, 800);
    lamichhane_frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    lamichhane_frame.setVisible(true);
}
}

```

Output:



6.2 Program to Implement 3D Scaling Through Origin

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class diwaskoscaling_3d extends JPanel {
    private int[][] orgCube = new int[8][2];
    private int[][] scaledCube = new int[8][2];
    public diwaskoscaling_3d(int[][] vertices, double sx, double sy) {
        for (int i = 0; i < 8; i++) {
            orgCube[i][0] = vertices[i][0];
            orgCube[i][1] = vertices[i][1];
        }
        for (int i = 0; i < 8; i++) {
            scaledCube[i][0] = (int) Math.round(vertices[i][0] * sx);
            scaledCube[i][1] = (int) Math.round(vertices[i][1] * sy);
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, height / 2, width, height / 2);
        g.drawLine(width / 2, 0, width / 2, height);
        int xOffset = width / 2;
        drawCube(g, orgCube, Color.BLACK, "Original", "Cube 1", xOffset);
        drawCube(g, scaledCube, new Color(0, 128, 255), "Scaled", "Cube 2", xOffset);
    }
    private void drawCube(Graphics g, int[][] cube, Color color, String label, String name, int
xOffset) {
        g.setColor(color);
        drawLine(g, cube, 0, 1, xOffset);
        drawLine(g, cube, 1, 2, xOffset);
        drawLine(g, cube, 2, 3, xOffset);
        drawLine(g, cube, 3, 0, xOffset);
        drawLine(g, cube, 4, 5, xOffset);
        drawLine(g, cube, 5, 6, xOffset);
        drawLine(g, cube, 6, 7, xOffset);
        drawLine(g, cube, 7, 4, xOffset);
        for (int i = 0; i < 4; i++) {
            drawLine(g, cube, i, i + 4, xOffset);
        }
    }
}
```



```

        for (int i = 0; i < 8; i++) {
            g.fillOval(
                cube[i][0] + xOffset - 4,
                flipY(cube[i][1]) - 4,
                8, 8
            );
        }
        g.setFont(new Font("Arial", Font.PLAIN, 14));
        g.drawString(label,
            cube[0][0] + xOffset - 20,
            flipY(cube[0][1]) - 20
        );
        g.setFont(new Font("Arial", Font.BOLD, 16));
    }
    private void drawLine(Graphics g, int[][] cube, int i, int j, int xOffset) {
        g.drawLine(
            cube[i][0] + xOffset, flipY(cube[i][1]),
            cube[j][0] + xOffset, flipY(cube[j][1])
        );
    }
    private int flipY(int y) {
        return getHeight() / 2 - y;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] vertices = new int[8][2];
        System.out.println("Enter 8 vertices of cube: ");
        for (int i = 0; i < 8; i++) {
            System.out.print("Vertex " + (i + 1) + ": ");
            vertices[i][0] = sc.nextInt();
            vertices[i][1] = sc.nextInt();
        }
        System.out.print("Enter scaling factor in x (sx): ");
        double sx = sc.nextDouble();
        System.out.print("Enter scaling factor in y (sy): ");
        double sy = sc.nextDouble();
        JFrame frame = new JFrame("DIWAS LAMICHHANE --> 3D Scaling of Cube");
        frame.add(new diwaskoscaling_3d(vertices, sx, sy));
        frame.setSize(1200, 800);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Output:

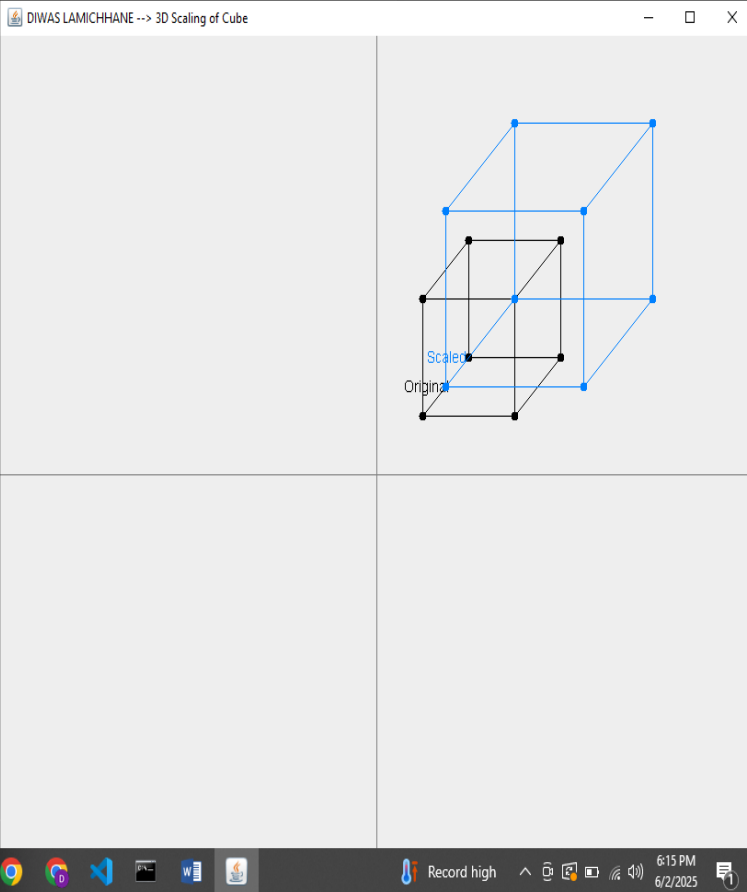
```
Command Prompt - java diwaskoscaling_3d
Microsoft Windows [Version 10.0.19045.5796]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Abhi's PC>cd desktop

C:\Users\Abhi's PC\Desktop>cd CGPractical

C:\Users\Abhi's PC\Desktop\CGPractical>javac diwaskoscaling_3d.java

C:\Users\Abhi's PC\Desktop\CGPractical>java diwaskoscaling_3d
Enter 8 vertices of cube:
Vertex 1: 50 50
Vertex 2: 150 50
Vertex 3: 150 150
Vertex 4: 50 150
Vertex 5: 100 100
Vertex 6: 200 100
Vertex 7: 200 200
Vertex 8: 100 200
Enter scaling factor in x (sx): 1.5
Enter scaling factor in y (sy): 1.5
```



6.3 Program to Implement 3D Rotation

Source Code:

```
import javax.swing.*.*;
import java.awt.*.*;
import java.util.Scanner;

public class diwaskoRotation_3d extends JPanel {
    private int[][] diwas_cube1 = new int[8][2];
    private int[][] diwas_cube2 = new int[8][2];
    public diwaskoRotation_3d(int[][] lamichhane_vertices, double rotAngle) {
        for (int i = 0; i < 8; i++) {
            diwas_cube1[i][0] = lamichhane_vertices[i][0];
            diwas_cube1[i][1] = lamichhane_vertices[i][1];
        }
        for (int i = 0; i < 8; i++) {
            diwas_cube2[i] = lamichhane_rotatePoint(lamichhane_vertices[i], rotAngle);
        }
    }
    private int[] lamichhane_rotatePoint(int[] point, double angleDegrees) {
        double angleRad = Math.toRadians(-angleDegrees);
        int x = point[0];
        int y = point[1];
        int xr = (int) Math.round(x * Math.cos(angleRad) - y * Math.sin(angleRad));
        int yr = (int) Math.round(x * Math.sin(angleRad) + y * Math.cos(angleRad));
        return new int[]{xr, yr};
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, height / 2, width, height / 2);
        g.drawLine(width / 2, 0, width / 2, height);
        int xOffset = width / 2;
        lam_drawCube(g, diwas_cube1, Color.BLACK, "Original", "CUBE 1", xOffset);
        lam_drawCube(g, diwas_cube2, new Color(150, 75, 0), "Rotated", "Cube 2", xOffset);
    }
    private void lam_drawCube(Graphics g, int[][] ba_cube, Color lam_edgeColor, String ab_label,
String ab_name, int xOffset) {
        g.setColor(lam_edgeColor);
        lam_drawLine(g, ba_cube, 0, 1, xOffset);
        lam_drawLine(g, ba_cube, 1, 2, xOffset);
        lam_drawLine(g, ba_cube, 2, 3, xOffset);
        lam_drawLine(g, ba_cube, 3, 0, xOffset);
    }
}
```

```

lam_drawLine(g, ba_cube, 4, 5, xOffset);
lam_drawLine(g, ba_cube, 5, 6, xOffset);
lam_drawLine(g, ba_cube, 6, 7, xOffset);
lam_drawLine(g, ba_cube, 7, 4, xOffset);
for (int i = 0; i < 4; i++) {
    lam_drawLine(g, ba_cube, i, i + 4, xOffset);
}
for (int i = 0; i < 8; i++) {
    g.fillOval(
        ba_cube[i][0] + xOffset - 4,
        flipY(ba_cube[i][1]) - 4,
        8, 8);
}
g.setFont(new Font("Arial", Font.PLAIN, 14));
g.drawString(ab_label,
    ba_cube[0][0] + xOffset - 20,
    flipY(ba_cube[0][1]) - 20);
g.setFont(new Font("Arial", Font.BOLD, 16));
g.drawString(ab_name, ba_cube[3][0] + xOffset, flipY(ba_cube[3][1]) + 60);}
private void lam_drawLine(Graphics g, int[][] ba_cube, int diw_i, int diw_j, int xOffset) {
    g.drawLine(
        ba_cube[diw_i][0] + xOffset, flipY(ba_cube[diw_i][1]),
        ba_cube[diw_j][0] + xOffset, flipY(ba_cube[diw_j][1])
    );
}
private int flipY(int y) {
    return getHeight() / 2 - y;
}
public static void main(String[] args) {
    Scanner lamichhane_scanner = new Scanner(System.in);
    int[][] lamichhane_vertices = new int[8][2];
    System.out.println("Enter 8 vertices of the cube: ");
    for (int i = 0; i < 8; i++) {
        System.out.print("Vertex " + (i + 1) + ": ");
        lamichhane_vertices[i][0] = lamichhane_scanner.nextInt();
        lamichhane_vertices[i][1] = lamichhane_scanner.nextInt();
    }
    System.out.print("Enter rotation angle: ");
    double rotationAngle = lamichhane_scanner.nextDouble();
    JFrame lamichhane_frame = new JFrame("DIWAS LAMICHHANE --> 3D Cube Rotation");
    diwaskoRotation_3d lamichhane_panel = new
    diwaskoRotation_3d(lamichhane_vertices, rotationAngle);
    lamichhane_frame.add(lamichhane_panel);
    lamichhane_frame.setSize(1200, 800);
}

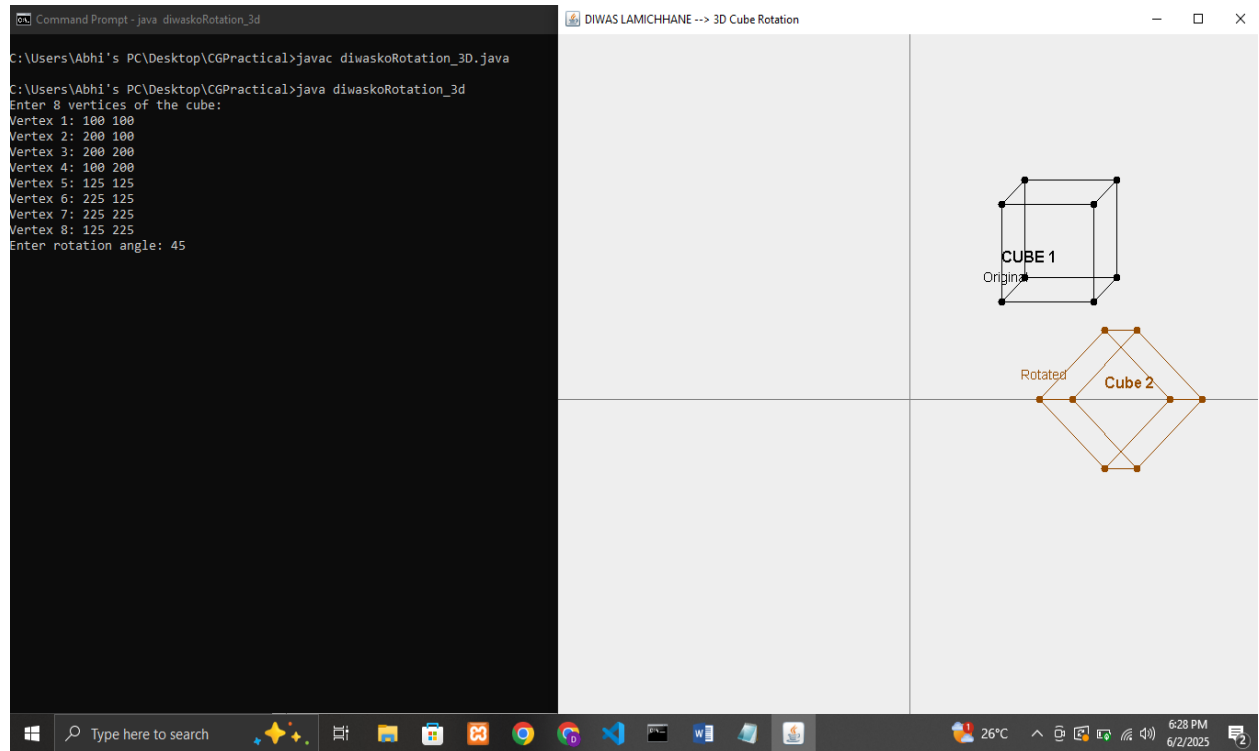
```

```

        lamichhane_frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        lamichhane_frame.setVisible(true);
    }
}

```

Output:



6.4 Program to Implement 3D Reflection Through X-axis

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class reflectionX_3D_iwas extends JPanel {
    private int[][] originalCube = new int[8][2];
    private int[][] reflectedCube = new int[8][2];
    public reflectionX_3D_iwas(int[][] vertices) {
        for (int i = 0; i < 8; i++) {
            originalCube[i][0] = vertices[i][0];
            originalCube[i][1] = vertices[i][1];
        }
        for (int i = 0; i < 8; i++) {
            reflectedCube[i][0] = vertices[i][0];
            reflectedCube[i][1] = -vertices[i][1];
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, height / 2, width, height / 2);
        g.drawLine(width / 2, 0, width / 2, height);
        int xOffset = width / 2;
        drawCube(g, originalCube, Color.BLACK, "Original Cube", "CUBE 1", xOffset);
        drawCube(g, reflectedCube, Color.MAGENTA, "Reflected X", "CUBE RX", xOffset);
    }
    private void drawCube(Graphics g, int[][] cube, Color color, String label, String name, int
xOffset) {
        g.setColor(color);
        drawLine(g, cube, 0, 1, xOffset);
        drawLine(g, cube, 1, 2, xOffset);
        drawLine(g, cube, 2, 3, xOffset);
        drawLine(g, cube, 3, 0, xOffset);
        drawLine(g, cube, 4, 5, xOffset);
        drawLine(g, cube, 5, 6, xOffset);
        drawLine(g, cube, 6, 7, xOffset);
        drawLine(g, cube, 7, 4, xOffset);
        for (int i = 0; i < 4; i++) {
            drawLine(g, cube, i, i + 4, xOffset);
        }
    }
}
```

```

        for (int i = 0; i < 8; i++) {
            g.fillOval(
                cube[i][0] + xOffset - 4,
                flipY(cube[i][1]) - 4,
                8, 8
            );
        }
        g.setFont(new Font("Arial", Font.PLAIN, 14));
        g.drawString(label,
            cube[0][0] + xOffset - 20,
            flipY(cube[0][1]) - 20
        );
        g.setFont(new Font("Arial", Font.BOLD, 16));
    }
    private void drawLine(Graphics g, int[][] cube, int i, int j, int xOffset) {
        g.drawLine(
            cube[i][0] + xOffset, flipY(cube[i][1]),
            cube[j][0] + xOffset, flipY(cube[j][1])
        );
    }
    private int flipY(int y) {
        return getHeight() / 2 - y;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] vertices = new int[8][2];
        System.out.println("Enter 8 vertices of the cube:");
        for (int i = 0; i < 8; i++) {
            System.out.print("Vertex " + (i + 1) + ": ");
            vertices[i][0] = sc.nextInt();
            vertices[i][1] = sc.nextInt();
        }
        JFrame frame = new JFrame("Diwas --> 3D Cube Reflection on X-axis ");
        frame.add(new reflectionX_3D_iwas(vertices));
        frame.setSize(1200, 800);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

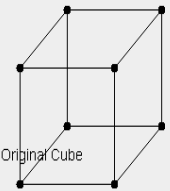
Output:

```
C:\Users\Abhi's PC\Desktop\CGPractical>javac reflectionX_3D_iwas.java

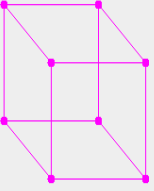
C:\Users\Abhi's PC\Desktop\CGPractical>java reflectionX_3D_iwas
Enter 8 vertices of the cube:
Vertex 1: 50 50
Vertex 2: 150 50
Vertex 3: 150
smkj
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:964)
    at java.base/java.util.Scanner.next(Scanner.java:1619)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2284)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2238)
    at reflectionX_3D_iwas.main(reflectionX_3D_iwas.java:72)

C:\Users\Abhi's PC\Desktop\CGPractical>java reflectionX_3D_iwas
Enter 8 vertices of the cube:
Vertex 1: 50 50
Vertex 2: 150 50
Vertex 3: 150 150
Vertex 4: 50 150
Vertex 5: 100 100
Vertex 6: 200 100
Vertex 7: 200 200
Vertex 8: 100 200
```

Diwas --> 3D Cube Reflection on X-axis



Original Cube



Reflected X

6.5 Program to Implement 3D Reflection Through Y-axis

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

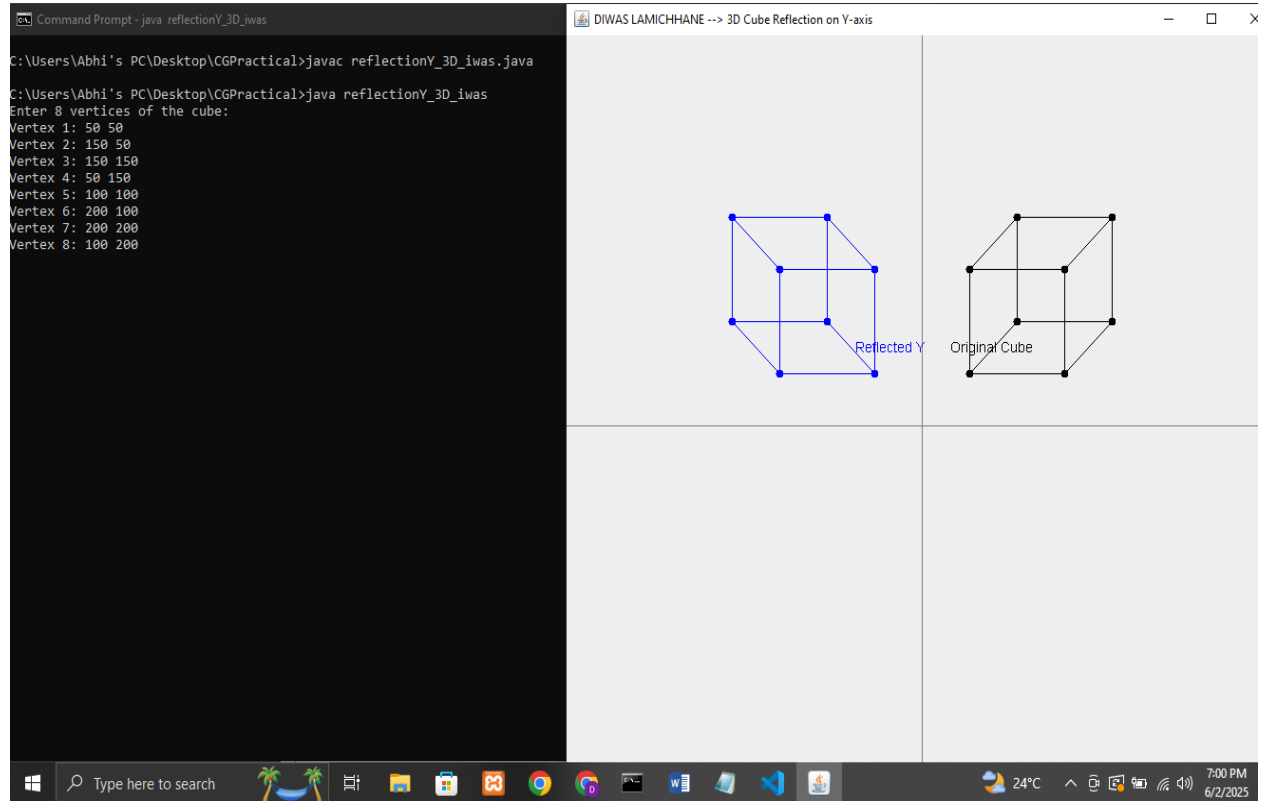
public class reflectionY_3D_iwas extends JPanel {
    private int[][] originalCube = new int[8][2];
    private int[][] reflectedCube = new int[8][2];
    public reflectionY_3D_iwas(int[][] vertices) {
        for (int i = 0; i < 8; i++) {
            originalCube[i][0] = vertices[i][0];
            originalCube[i][1] = vertices[i][1];
        }
        for (int i = 0; i < 8; i++) {
            reflectedCube[i][0] = -vertices[i][0];
            reflectedCube[i][1] = vertices[i][1];
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, height / 2, width, height / 2);
        g.drawLine(width / 2, 0, width / 2, height);
        int xOffset = width / 2;
        drawCube(g, originalCube, Color.BLACK, "Original Cube", "CUBE 1", xOffset);
        drawCube(g, reflectedCube, Color.BLUE, "Reflected Y", "CUBE RY", xOffset);
    }
    private void drawCube(Graphics g, int[][] cube, Color color, String label, String name, int
xOffset) {
        g.setColor(color);
        drawLine(g, cube, 0, 1, xOffset);
        drawLine(g, cube, 1, 2, xOffset);
        drawLine(g, cube, 2, 3, xOffset);
        drawLine(g, cube, 3, 0, xOffset);
        drawLine(g, cube, 4, 5, xOffset);
        drawLine(g, cube, 5, 6, xOffset);
        drawLine(g, cube, 6, 7, xOffset);
        drawLine(g, cube, 7, 4, xOffset);
        for (int i = 0; i < 4; i++) {
            drawLine(g, cube, i, i + 4, xOffset);
        }
    }
}
```

```

        for (int i = 0; i < 8; i++) {
            g.fillOval(
                cube[i][0] + xOffset - 4,
                flipY(cube[i][1]) - 4,
                8, 8
            );
        }
        g.setFont(new Font("Arial", Font.PLAIN, 14));
        g.drawString(label,
            cube[0][0] + xOffset - 20,
            flipY(cube[0][1]) - 20
        );
        g.setFont(new Font("Arial", Font.BOLD, 16));
    }
    private void drawLine(Graphics g, int[][] cube, int i, int j, int xOffset) {
        g.drawLine(
            cube[i][0] + xOffset, flipY(cube[i][1]),
            cube[j][0] + xOffset, flipY(cube[j][1])
        );
    }
    private int flipY(int y) {
        return getHeight() / 2 - y;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] vertices = new int[8][2];
        System.out.println("Enter 8 vertices of the cube:");
        for (int i = 0; i < 8; i++) {
            System.out.print("Vertex " + (i + 1) + ": ");
            vertices[i][0] = sc.nextInt();
            vertices[i][1] = sc.nextInt();
        }
        JFrame frame = new JFrame("DIWAS LAMICHHANE --> 3D Cube Reflection on Y-axis ");
        frame.add(new reflectionY_3D_iwas(vertices));
        frame.setSize(1200, 800);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Output:



6.6 Program to Implement 3D Shearing in XY Direction

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.util.Scanner;

public class shearingXY3D_DIWAS extends JPanel {
    private int[][] diwas_orgCube = new int[8][3];
    private int[][] diwas_shearedCube = new int[8][3];

    public shearingXY3D_DIWAS(int[][] lamichhane_vertices, double shx, double shy) {
        for (int i = 0; i < 8; i++) {
            diwas_orgCube[i][0] = lamichhane_vertices[i][0];
            diwas_orgCube[i][1] = lamichhane_vertices[i][1];
            diwas_orgCube[i][2] = lamichhane_vertices[i][2];
        }
        for (int i = 0; i < 8; i++) {
            diwas_shearedCube[i][0] = (int) Math.round(lamichhane_vertices[i][0] + shx *
lamichhane_vertices[i][2]);
            diwas_shearedCube[i][1] = (int) Math.round(lamichhane_vertices[i][1] + shy *
lamichhane_vertices[i][2]);
            diwas_shearedCube[i][2] = lamichhane_vertices[i][2];
        }
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        int width = getWidth();
        int height = getHeight();
        g.setColor(Color.GRAY);
        g.drawLine(0, height / 2, width, height / 2);
        g.drawLine(width / 2, 0, width / 2, height);
        int xOffset = width / 2;
        // Draw original and sheared cubes with spacing
        drawCube(g, diwas_orgCube, Color.BLACK, "Original Cube", xOffset - 200);
        drawCube(g, diwas_shearedCube, Color.RED, "Sheared XY", xOffset + 200);
    }

    private void drawCube(Graphics g, int[][] cube, Color color, String label, int xOffset) {
        g.setColor(color);
        int[][] edges = {
            {0,1},{1,2},{2,3},{3,0},
            {4,5},{5,6},{6,7},{7,4},
            {0,4},{1,5},{2,6},{3,7}
        };
    }
}
```

```

    for (int[] edge : edges) {
        drawLine(g, cube, edge[0], edge[1], xOffset);
    }
    for (int i = 0; i < 8; i++) {
        int[] p = project(cube[i][0], cube[i][1], cube[i][2]);
        g.fillOval(p[0] + xOffset - 4, flipY(p[1]) - 4, 8, 8);
    }
    g.setFont(new Font("Arial", Font.BOLD, 14));
    int[] labelPos = project(cube[0][0], cube[0][1], cube[0][2]);
    g.drawString(label, labelPos[0] + xOffset - 30, flipY(labelPos[1]) - 10);
}
private void drawLine(Graphics g, int[][] cube, int i, int j, int xOffset) {
    int[] p1 = project(cube[i][0], cube[i][1], cube[i][2]);
    int[] p2 = project(cube[j][0], cube[j][1], cube[j][2]);

    g.drawLine(
        p1[0] + xOffset, flipY(p1[1]),
        p2[0] + xOffset, flipY(p2[1])
    );
}
private int flipY(int y) {
    return getHeight() / 2 - y;
}
private int[] project(int x, int y, int z) {
    return new int[] {
        x + (int)(z * 0.5),
        y - (int)(z * 0.5)
    };
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int[][] lamichhane_vertices = new int[8][3];

    System.out.println("Enter 8 vertices of the cube (x y z):");
    for (int i = 0; i < 8; i++) {
        System.out.print("Vertex " + (i + 1) + ": ");
        lamichhane_vertices[i][0] = sc.nextInt();
        lamichhane_vertices[i][1] = sc.nextInt();
        lamichhane_vertices[i][2] = sc.nextInt();
    }
    System.out.print("Enter shearing factor in x (shx): ");
    double shx = sc.nextDouble();
    System.out.print("Enter shearing factor in y (shy): ");
    double shy = sc.nextDouble();
}

```

```

JFrame frame = new JFrame("@DIWAS LAMICHHANE - 3D Cube Shearing in XY Direction");
frame.add(new shearingXY3D_DIWAS(lamichhane_vertices, shx, shy));
frame.setSize(1200, 800);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Output:

